ISTITUTO TECNOLOGICO SUPERIORE ACADEMY

PER LE TECNOLOGIE DELLA INFORMAZIONE E DELLA COMUNICAZIONE

DEL PIEMONTE

Sede legale: Torino, Piazza Carlo Felice 18

DENOMINAZIONE CORSO : **Tecnico superiore System Administrator - CLOUD SPECIALIST**

Migrazione Cloud-Native e Data Automation : To-Do List su Amazon Web Services.

Candidato

TOMA DELIA ROBERTA

Indice

1. Introduzione....4
1.1 Obiettivi del project work....4
1.2 Presentazione dell'azienda: Present gruppo Exprivia....4
1.3 Panoramica del progetto: la To-Do List....4
2. Tecnologie e Strumenti Utilizzati....5
2.1 Ambiente di sviluppo....5
2.2 Containerizzazione: Docker e Docker Desktop....5
2.3 Ecosistema Cloud: Panoramica dei servizi AWS selezionati....6
3. Analisi dell'applicazione in locale....8
3.1 Persistenza dei dati: SQL Server locale....8
3.2 Architettura backend: .NET Core e Web API....9
3.3 Testing e validazione: Swagger e Postman....10
3.4 Gestione dell'autenticazione: Logica token e cookie....10
3.5 Frontend dinamico: Javascript, Handlebars e Bootstrap....12
4. Progettazione dell'infrastruttura Cloud (AWS)....14
4.1 Disegno della rete: VPC, Subnet e Security Groups....14
4.2 Migrazione del database (RDS)....16
4.3 Strategia di sicurezza: IAM Roles e il principio di "Least Privilege"....18
5. Containerizzazione (Il backend)....19
5.1 Creazione dell'immagine Docker e storage su Amazon ECR....19
5.2 Deployment su ECS Fargate: Task Definition e Service....20
5.3 Esposizione delle API: Application Load Balancer e API Gateway....21
6. Distribuzione Frontend e sicurezza avanzata....25
6.1 Hosting statico su S3 e distribuzione globale tramite CloudFront....25
6.2 Protezione degli accessi: Origin Access Control (OAC)....26
6.3 Gestione dei CORS (Cross-Origin Resource Sharing)....26
7. Modulo Analytics e Data Automation (Serverless)....29
7.1 Statistiche admin in tempo reale....29
7.2 Architettura Event-Driven: AWS Lambda e EventBridge Scheduler....30
7.3 Integrazione frontend: visualizzazione dati tramite chart.js....31

7.4 Ottimizzazione dei costi e performance tramite salvataggio su S3.....33
8. Evoluzione DevOps e automazione del ciclo di vita.....34
8.1 Automazione del flusso CI/CD.....34
8.2 Build Specification e Tracciabilità.....36
8.3 Deployment e Zero-Downtime.....37
9. Conclusioni e Sviluppi Futuri.....38
9.1 Analisi dei risultati ottenuti.....38
9.2 Evoluzione verso l'Infrastructure as Code (IaC).....38
9.3 Monitoraggio e osservabilità Avanzata.....39
9.4 Considerazioni personali.....39
10. Glossario termini tecnici.....41
11. Bibliografia e Sitografia.....45

2 - Tecnologie e Strumenti Utilizzati

Per la realizzazione dell'intero ecosistema Cloud-Native[10], è stata selezionata una suite di strumenti professionali che spaziano dallo sviluppo di codice alla containerizzazione, fino alla gestione dell'infrastruttura cloud. La scelta di queste tecnologie è stata guidata dalla necessità di garantire scalabilità, portabilità e sicurezza.

2.1 Ambiente di sviluppo

Lintegrazione di diversi linguaggi e ambienti di sviluppo ha permesso di gestire ogni layer dell'applicazione con lo strumento più idoneo:

- **Visual Studio 2022**: È stato l'IDE (Integrated Development Environment) di riferimento per la parte di backend. Grazie al supporto nativo per il framework .NET 8, ha facilitato la creazione delle Web API, la gestione dei pacchetti NuGet e il debugging.

FIGURA 1: LOGO VISUAL STUDIO

- **Visual Studio Code**: Utilizzato come editor leggero e versatile per lo sviluppo del frontend (HTML, CSS e Javascript).

FIGURA 2: LOGO VISUAL STUDIO CODE

- **Linguaggi di programmazione**:
  - *C#*: Elemento principale del backend, scelto per la robustezza e le performance elevate nella gestione di richieste asincrone.

FIGURA 3: LOGO C#

  - *Javascript*: Utilizzato nel frontend per gestire la dinamicità dell'interfaccia e la comunicazione asincrona con le API tramite Fetch API.

FIGURA 4: LOGO JAVASCRIPT

  - *Python*: Adottato per lo sviluppo della AWS Lambda nel modulo Analytics, grazie alla sua sintassi snella e all'ampia disponibilità di librerie per la manipolazione dei dati.

FIGURA 5: LOGO PYTHON

2.2 Containerizzazione: Docker e Docker Desktop

La tecnologia dei container è stata l'elemento di collegamento tra l'ambiente di sviluppo locale e il cloud.

- **Docker**: È stato utilizzato per pacchettizzare l'applicazione .NET insieme a tutte le sue dipendenze, variabili d'ambiente e runtime. Questo garantisce che il

software si comporti in modo identico ovunque venga eseguito, eliminando il problema dell'incopatibilità.

- **Docker Desktop**: Ha permesso di gestire il ciclo di vita dei container localmente, consentendo di buildare le immagini, testarle e verificarne l 'esposizione delle porte prima del caricamento definitivo su Amazon ECR.

FIGURA 6: LOGO DOCKER DESKTOP

2.3 Ecosistema Cloud: Panoramica dei servizi AWS selezionati

L'architettura poggia interamente su Amazon Web Services (AWS). Per garantire alta disponibilità e sicurezza, sono stati selezionati servizi gestiti che seguono i principi del Cloud Computing:

FIGURA 7: LOGO AMAZON WEB SERVICES

- **Compute**:
  - *Fargate*: Motore di calcolo serverless per container che permette di eseguire i task di ECS senza dover gestire o patchare i server sottostanti.

    FIGURA 8: AWS FARGATE
  - *Lambda*: Servizio di calcolo event-driven utilizzato per eseguire codice in risposta a trigger specifici, ideale per l'automazione delle statistiche.

    FIGURA 9: AWS LAMBDA
  - *ECS* (Elastic Container Service): Servizio di orchestrazione di container altamente scalabile che gestisce l'esecuzione dei task Docker.

    FIGURA 10: AWS ELASTIC CONTAINER SERVICE
- **Networking**:
  - *Virtual Private Cloud* (VPC): Rete virtuale logicamente isolata che permette di segregare le risorse e definire regole di instradamento del traffico.

    FIGURA 11: AWS VIRTUAL PRIVATE CLOUD
  - *Application Load Balancer* (ALB): Bilanciatore di carico che distribuisce il traffico verso i container (che nel caso specifico è stato utilizzato soltanto uno) del backend, garantendo la resilienza.

    FIGURA 12: AWS APPLICATION LOAD BALANCER
  - *CloudFront*: Rete per la distribuzione dei contenuti (CDN) che si occupa della consegna del frontend statico agli utenti e fornisce un layer di sicurezza aggiuntivo.

    FIGURA 13: AWS CLOUDFRONT

- *API Gateway*: Servizio completamente gestito che facilita la creazione, la pubblicazione e la protezione delle API, fungendo da "porta d'ingresso" per le richieste verso il backend.

FIGURA 14: AWS API GATEWAY

- **Storage, Database e Registry**:
  - *S3* (Simple Storage Service): Servizio di storage a oggetti utilizzato per l'hosting dei file statici, l'Origin Access Control e il backup dei dati.

FIGURA 15: AWS BUCKET S3

  - *Relational Database Service* (RDS): Servizio gestito per database relazionali, utilizzato per ospitare l'istanza SQL Server in modo sicuro.

FIGURA 16: AWS RELATIONAL DATABASE SERVICE

  - *Elastic container registry* (ECR): Registro Docker completamente gestito che permette di archiviare, gestire e distribuire le immagini dei container.

FIGURA 17: AWS ELASTIC CONTAINER REGISTRY

- **Security & Monitoring**:
  - *Identity and Access Management (IAM)*: Servizio per la gestione granulare degli accessi e dei permessi tra i vari componenti dell'infrastruttura.

FIGURA 18: AWS IDENTITY AND ACCESS MANAGEMENT

  - *CloudWatch*: Strumento di monitoraggio centralizzato per la raccolta dei log applicativi e l'analisi delle metriche di performance.

FIGURA 19: AWS CLOUDWATCH

- **DevOps & CI/CD (Continuous Integration / Continuous Deployment):**
  - *CodePipeline*: Servizio di orchestrazione che automatizza le fasi di rilascio del software. Gestisce il flusso del codice dal repository su GitHub fino al deployment finale sui container ECS.

FIGURA 20: AWS CODEPIPELINE

  - *CodeBuild*: Servizio di compilazione completamento gestito che esegue i test e pachetizza il codice sorgente in immagini Docker pronte per essere archiviate su ECR.

FIGURA 21: AWS CODEBUILD

  - *Webhook*: Meccanismo di notifica in tempo reale che funge da trigger: ogni volta che viene effettuato un push del codice sul repository, il Webhook avvisa CodePipeline di far partire automaticamente il processo di aggiornamento.

3 - Analisi dell'applicazione in locale

In una prima fase di sviluppo fino allo unit test le applicazioni sono state avviate in locale (on premise[11]) per poi essere integrate nell'architettura cloud per il test finale.

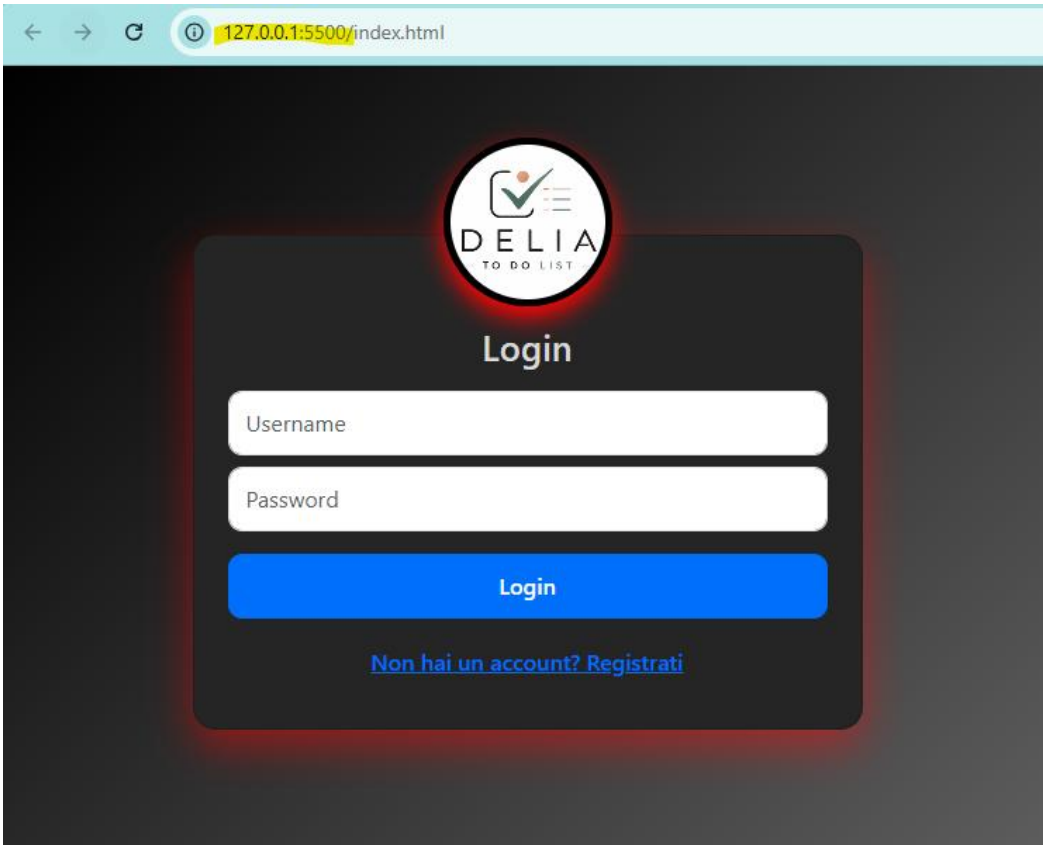


FIGURA 22: PAGINA INIZIALE DI LOGIN DEL SITO IN LOCALE

3.1 Persistenza dei dati: SQL Server locale

Per la memorizzazione e la gestione dei dati è stato utilizzato Microsoft SQL Server. La base di dati, denominata *ToDoDB*, è stata progettata per garantire l'integrità dei dati e facilitare le operazioni di manutenzione.

Il database si articola su due tabelle principali, legate da una relazione uno-a-molti[12]:

- **Users**:
  - *Id* → primary key[13]
  - *Username* → identificatore testuale dell'utente
  - *PasswordHash* → è stata implementata una logica di hashing
  - *IsAdmin* → campo booleano[14] che definisce se un utente è admin o user normale

- *Token* → stringa casuale generata al login, utilizzata dall'AuthFilter per validare le richieste API

Risultati | Messaggi

	Id	Username	PasswordHash	IsAdmin	Token
1	1	delia	1234	1	c72f153b-eb9b-4164-b78c-5f9e452681c6
2	2	marco88	1234	0	
3	3	claudia_green	1234	0	
4	4	stefano_run	1234	0	

FIGURA 23: TABELLA USERS

- **Items**:
  - *Id* → chiave primaria delle task
  - *Task* → contenuto testuale della nota
  - *IsCompleted* → flag booleano per distinguere i task completati da quelli in corso
  - *CreatedAt* → timestamp di creazione
  - *UserId* → foreign key[15] collegata alla tabella Users

Risultati | Messaggi

	Id	Task	IsCompleted	CreatedAt	UserId
1	37	Comprare caffè e zucchero	0	2026-02-27 11:46:08.550	1
2	38	Prenotare tavolo per cena sabato	1	2026-02-27 11:46:15.610	1
3	39	Ricordarsi di caricare la tesi su Drive	0	2026-02-27 11:46:21.480	1
4	40	Portare la macchina a lavare	1	2026-02-27 11:46:53.067	2
5	41	Comprare croccantini per il gatto	1	2026-02-27 11:47:00.393	2
6	42	Andare a correre al parco	0	2026-02-27 11:47:22.490	3
7	43	Ritirare pacco in posta	1	2026-02-27 11:47:27.920	3
8	45	Studiare per l'esame di lunedì	0	2026-02-27 11:48:00.167	4
9	46	Chiamare la nonna per gli auguri	1	2026-02-27 11:48:05.947	4

FIGURA 24: TABELLA "ITEMS" CON ESEMPI DI TASK

3.2 Architettura backend: .NET Core e Web API

Il cuore logico dell'applicazione è stato realizzato utilizzando il framework .NET Core. La scelta è ricaduta su un'architettura di tipo Web API, che permette di separare nettamente la logica di elaborazione dati dall'interfaccia utente, facilitando così la futura scalabilità in ambiente containerizzato.

Il backend gestisce le richieste HTTP provenienti dal client e interagisce con il database attraverso Entity Framework Core[16], un ORM (Object-Relational Mapper[17]) che ha

permesso di mappare le classi C# direttamente sulle tabelle del database SQL Server. Ogni endpoint dell'API è stato progettato per essere "stateless[18]", delegando la gestione della sessione al sistema di token (descritto in capitolo 3.4).

3.3 Testing e Validazione: Swagger e Postman

Per garantire l'affidabilità delle API prima dell'integrazione finale, è stata adottata la seguente metodologia di test:

- **Swagger**: Integrato direttamente nel middleware di .NET, ha permesso di avere una documentazione interattiva e visuale di tutti gli endpoint (GET, POST, PUT, DELETE), consentendo test rapidi.
- **Postman**: Utilizzato per simulare chiamate complesse, testare l'invio dei token e verificare il comportamento del server in caso di errori (es. credenziali errate o token scaduti). Questo passaggio è stato cruciale per validare la robustezza del sistema prima di sviluppare l'interfaccia Javascript.

3.4 Gestione dell'autenticazione: Logica Token e Cookie[19]

Uno degli aspetti più complessi dello sviluppo è stata la gestione della sicurezza. Per proteggere i dati degli utenti è stato implementato un sistema di autenticazione che intercetta ogni richiesta attraverso un sistema di filtraggio chiamato **AuthFilter**, che verrà poi richiamato prima del controller principale.

```
Backend                                                    ToDoContro
1   using Microsoft.AspNetCore.Mvc;
2   using Microsoft.EntityFrameworkCore;
3   using Backend.Models;
4   using Backend.Dtos;
5   using Amazon.S3;
6   using Amazon.S3.Model;
7
8   //prima di eseguire qualsiasi cosa del controller passa prima da
9   //AuthFilter
10  [ServiceFilter(typeof(AuthFilter))]
11  [ApiController]
12  [Route("api/todos")]
    1 riferimento
13  public class ToDoController : ControllerBase
14  {
```

FIGURA 25: RICHIAMO AUTHFILTER PRIMA DEL CONTROLLER (SE IL FILTRO FALLISCE NON PASSA AL CONTROLLER)

Token a stringa casuale: Invece di utilizzare soluzioni preconfigurate (come il JWT[20]), è stata adottata la scelta di implementare manualmente un sistema basato su token a stringa casuale. Al momento del login, se le credenziali fornite dall'utente sono corrette, il server genera una stringa alfanumerica univoca e sicura. Questa scelta metodologica è

stata perseguita per approfondire le logiche di funzionamento interno dei sistemi di sessione e il mantenimento dello stato tra diverse chiamate HTTP.

```
//stringa per generare un token unico
//GUID ha 128 bit di entropia
user.Token = Guid.NewGuid().ToString();

//salvo il token appena generato nel db
_context.SaveChanges();
```

FIGURA 26: STRINGA PER GENERARE UN TOKEN CASUALE CON 128 BIT DI ENTROPIA

Il token generato viene salvato in una colonna dedicata della tabella *Users* nel database.

	Id	Username	PasswordHash	IsAdmin	Token
1	1	delia	1234	1	c72f153b-eb9b-4164-b78c-5f9e452681c6

FIGURA 27: TABELLA USERS CON COLONNA TOKEN

Ad ogni chiamata API, l'*AuthFilter* recupera il token e interroga il database. Il sistema verifica se esiste un utente associato a quel token e ne recupera il Ruolo (User o Admin), permettendo così di filtrare i dati in base ai permessi: l'utente semplice visualizzerà solo i propri record, mentre l'amministratore avrà accesso alla visualizzazione globale.

```
public async Task OnActionExecutionAsync(
    ActionExecutingContext context,
    ActionExecutionDelegate next)
{
    if (context.HttpContext.Request.Method.ToUpper() == "OPTIONS")
    {
        await next();
        return;
    }
    //Leggo il token dai cookie
    var token = context.HttpContext.Request.Cookies["token"];

    if (string.IsNullOrEmpty(token))
    {
        //Nessun token -> non autenticato
        context.Result = new UnauthorizedResult();
        return;
    }

    //Cerco l'utente con quel token
    var user = _context.Users.FirstOrDefault(u => u.Token == token);

    if (user == null)
    {
        //Token non valido
        context.Result = new UnauthorizedResult();
        return;
    }

    //Salvo l'utente nel contesto della request
    context.HttpContext.Items["User"] = user;

    //Tutto ok → continua verso il controller
    await next();
}
```

FIGURA 28: LOGICA DI AUTHFILTER

Sicurezza e Cookie: Per proteggere il token dal rischio di furti tramite script malevoli (attacchi XSS[21]), la stringa non viene salvata nel localStorage[22] del browser, ma inviata tramite un cookie HTTP-Only. L'attributo HTTP-Only garantisce che il cookie sia accessibile solo dal browser durante la comunicazione con il server e non sia leggibile tramite Javascript. Inoltre, nel backend è stata configurata la gestione dei CORS (Cross-Origin Resource Sharing[23]) con l'opzione *AllowCredentials()*[24], impostazione fondamentale per permettere il passaggio sicuro dei cookie tra il dominio del frontend e quello del backend, specialmente in vista della futura pubblicazione CloudFront e API Gateway.

```
builder.Services.AddCors(options => {
    options.AddPolicy("AllowAll", policy => {
        policy.WithOrigins("https://d17vbcq73r96m2.cloudfront.net") //solo il sito cloudfront può chiamare l'API
            .AllowAnyMethod() //permette GET, POST, PUT, DELETE
            .AllowAnyHeader()//permette l'invio di intestazioni (es. Content-Type)
            .AllowCredentials(); // FONDAMENTALE per i cookie
    });
});
```

FIGURA 29: REGOLE DI CORS NEL BACKEND

3.5 Frontend dinamico: JavaScript, Handlebars e Bootstrap[25]

L'interfaccia utente è stata progettata puntando su reattività e pulizia estetica. Per lo sviluppo sono state integrate tre tecnologie principali:

- **Bootstrap**[25]: Per la componente stilistica è stato utilizzato il framework Bootstrap. Questo ha permesso di realizzare un'interfaccia responsive[26] (adattabile a diversi dispositivi). Attraverso le classi native (come container, card, d-none) è stata gestita la visibilità condizionale degli elementi senza dover cambiare pagina HTML.
- **Handlebars.js**: Invece di scrivere codice HTML ripetitivo, è stato definito uno *<script id="todo-template">* che funge da modello. Javascript compila questo template iniettandovi i dati JSON[27] ricevuti dal backend, gestendo automaticamente dati complessi come la barratura del testo per i task completati (*text-decoration-line-through*).

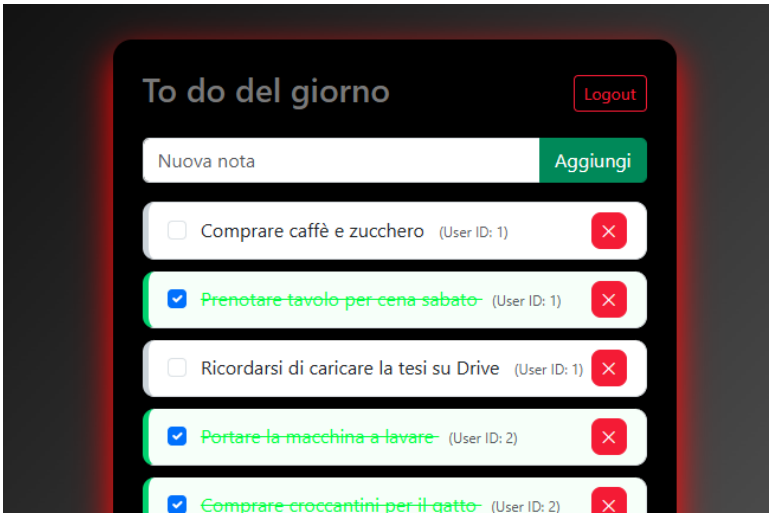


FIGURA 30: VISUALIZZAZIONE DELLE TODO DELL'ADMIN (VISUALIZZA TASK DI TUTTI GLI UTENTI)

- **Vanilla JavaScript:** Il file *script.js* orchestra l'intera logica applicativa. Utilizza le Fetch API[28] con l'opzione *credentials: "include"* per interagire con le Web API di AWS in modo asincrono, garantendo che l'utente non percepisca interruzioni durante l'uso dell'app

4 - Progettazione dell'infrastruttura Cloud (AWS)

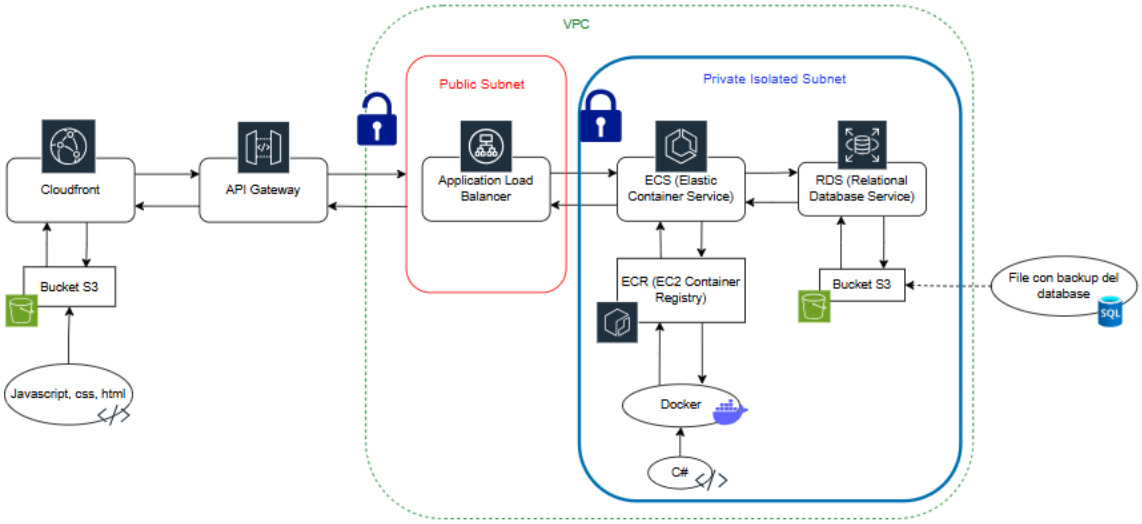


FIGURA 31: DISEGNO ARCHITETTURA DELLA WEB APP CON I VARI SERVIZI AWS UTILIZZATI

La migrazione dell'applicazione To-Do List ha richesto la progettazione di un'architettura Cloud-Native. L'obiettivo non è stato un semplice trasferimento di file, ma una re-ingegnerizzazione mirata a garantire isolamento, sicurezza e alta affidabilità attraverso i servizi gestiti di Amazon Web Services.

4.1 Disegno della rete: *VPC*, Subnet[29] e Security Groups

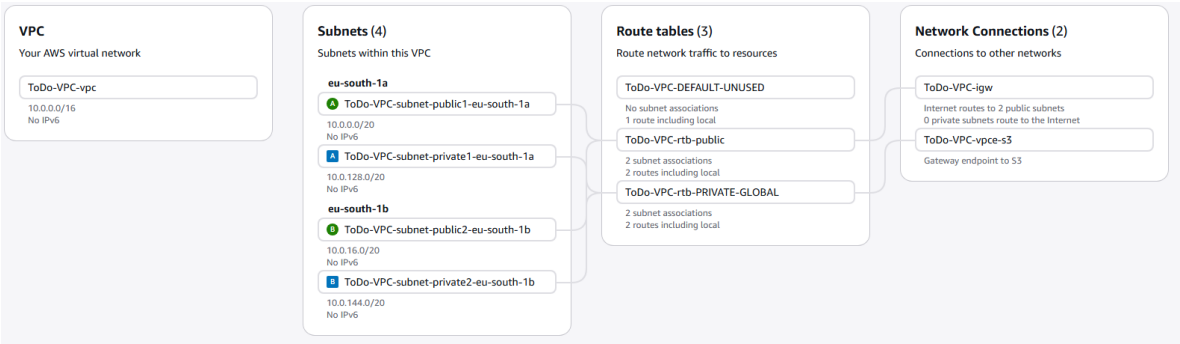


FIGURA 32: DISEGNO DELLA VPC CON LE VARIE SUBNET

La base dell'infrastruttura è la *VPC* (Virtual Private Cloud), una rete virtuale isolata che definisce il perimetro di sicurezza del progetto. La rete è suddivisa in *Subnet* con diverse finalità per separate logicamente i livelli dell'applicazione.

- **Subnet Pubbliche**: ospitano l'Application Load Balancer (ALB), agendo come punto di ingresso per il traffico proveniente da Internet.

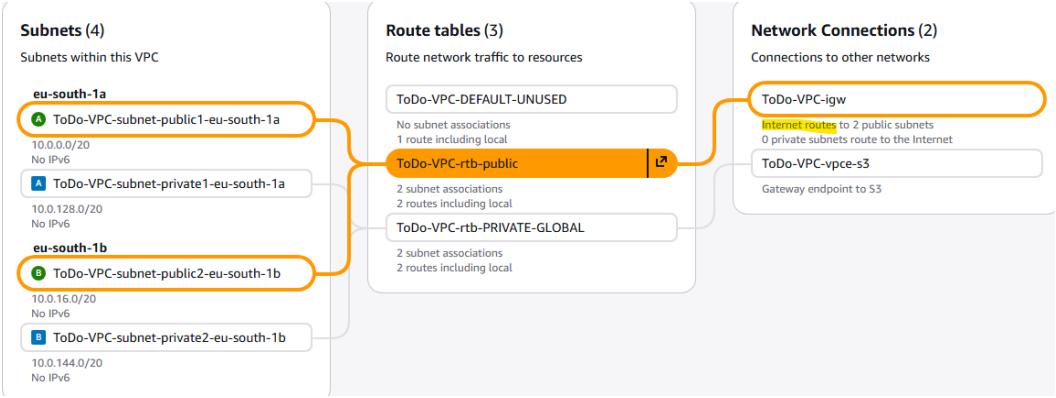


FIGURA 33: SUBNET PUBBLICHE COLLEGATE ALL'INTERNET GATEWAY

- **Subnet Private**: alloggiano le risorse critiche come il database RDS e i container del backend (ECS Fargate). Queste risorse sono prive di indirizzi IP pubblici e non sono raggiungibili direttamente dall'esterno, eliminando alla base il rischio di intrusioni dirette e proteggendo dati sensibili.

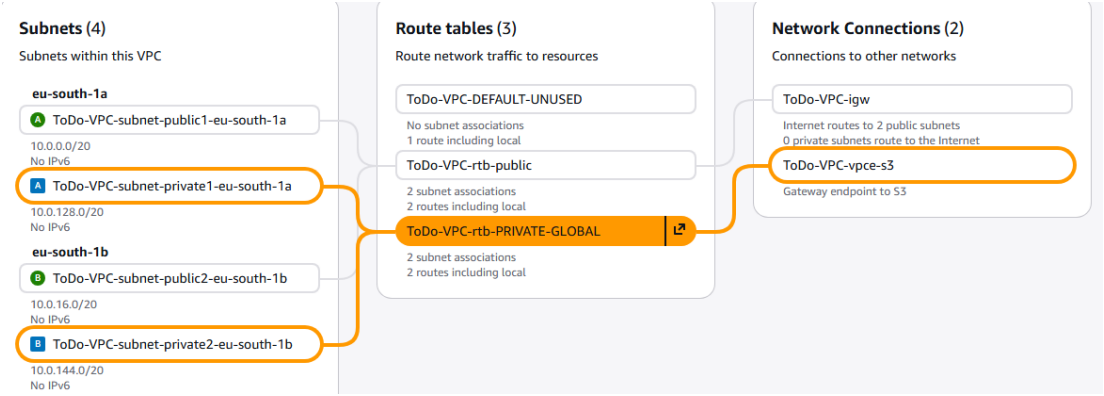
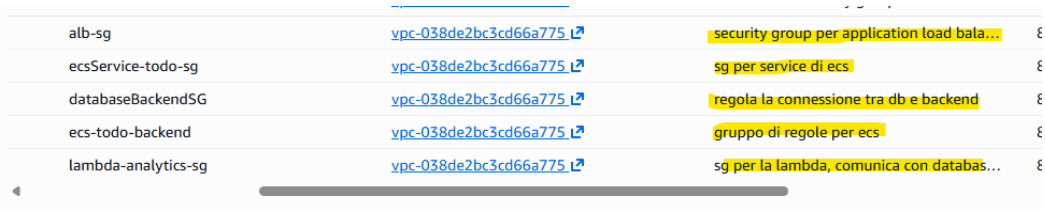


FIGURA 34: SUBNET PRIVATE, CHIUSE AD INTERNET E COLLEGATE SOLO AI BUCKET S3

- **Security Groups**: configurati come firewall virtuali a livello di istanza. È stata creata una comunicazione "a catena": l'Application Load Balancer accetta traffico dal web, il backend accetta traffico solo dall'ALB e il database RDS accetta connessioni solo dal backend sulla porta 1433. Questo isolamento garantisce che nessuna risorsa sia esposta oltre il necessario.



alb-sg	vpc-038de2bc3cd66a775	security group per application load bala...
ecsService-todo-sg	vpc-038de2bc3cd66a775	sg per service di ecs
databaseBackendSG	vpc-038de2bc3cd66a775	regola la connessione tra db e backend
ecs-todo-backend	vpc-038de2bc3cd66a775	gruppo di regole per ecs
lambda-analytics-sg	vpc-038de2bc3cd66a775	sg per la lambda, comunica con databas...

FIGURA 35: VARI SECURITY GROUPS UTILIZZATI PER RISORSE COME ECS, DATABASE, LOAD BALANCER E LAMBDA

4.2 Migrazione del Database (RDS)

Il passaggio al cloud ha richiesto la configurazione di un servizio database gestito che garantisse persistenza e sicurezza senza il bisogno della gestione hardw are. Per questo scopo è stato utilizzato **Amazon Relational Database Service**.

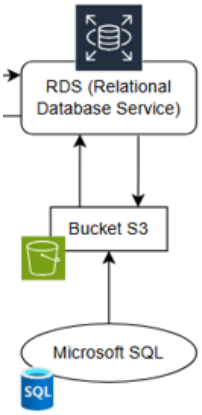


FIGURA 36: DATABASE NELL'INFRASTRUTTURA

Scelta dell'istanza: È stata selezionata un'istanza *Microsoft SQL Server Express Edition*, per mantenere la piena compatibilità con il codice .NET Core sviluppato in locale. L'istanza è stata configurata su una classe **t3.micro**[30], scelta ottimale per bilanciare costi e prestazioni durante la fase finale di sviluppo (rientrando nel *Free Tier* di AWS). Nonostante le dimensioni ridotte, l'istanza offre tutte le funzionalità necessarie alla To-Do List, inclusa la gestione dei backup automatizzati e il monitoraggio di *CloudWatch*.

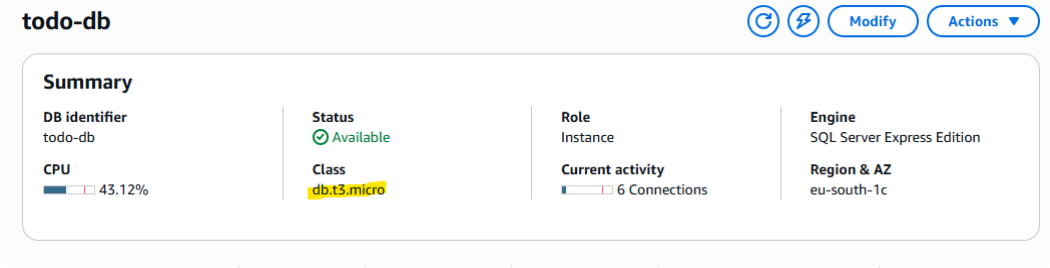


FIGURA 37: ISTANZA RDS IN FUNZIONE

Procedura di migrazione: La migrazione dei dati non è avvenuta tramite inserimento manuale, ma attraverso una procedura di Backup & Restore, articolata in 3 fasi:

1. **Esportazione locale**: Tramite SQL Server Management Studio (SSMS) è stato generato un file di backup con estensione *.bak* del database locale.

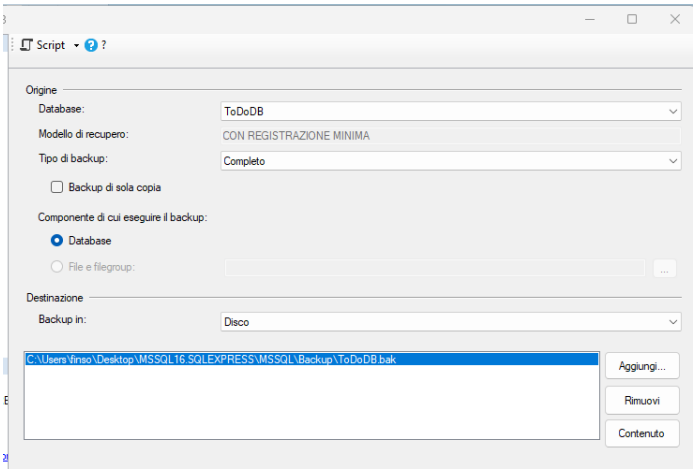


FIGURA 38: SALVATAGGIO FILE DI BACKUP (.BAK) DEL DATABASE LOCALE

2. **Storage su S3**: Il file è stato caricato su un bucket Amazon S3. Per permettere a RDS di leggere il file, è stato configurato un Option Group[31] specifico (citato antecedentemente).

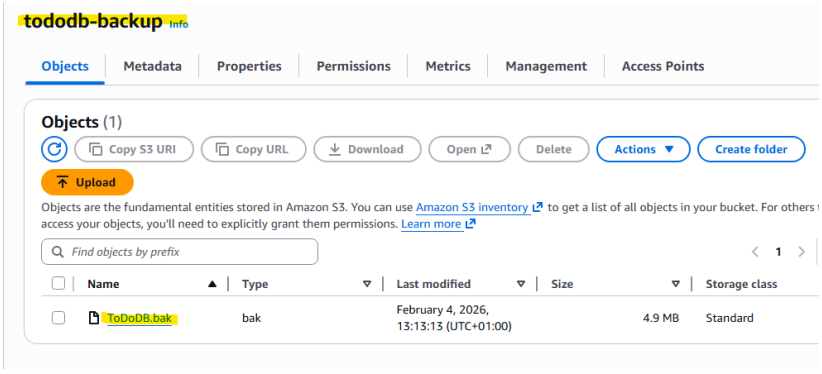


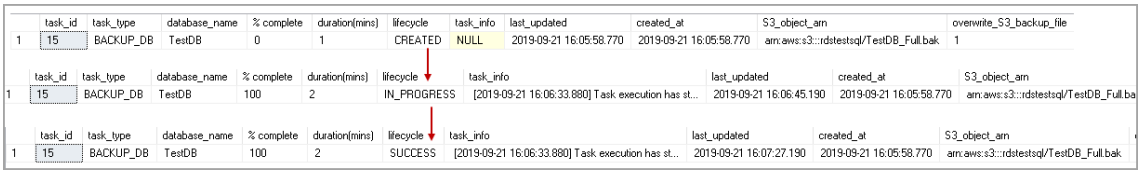
FIGURA 39: BUCKET S3 CON DENTRO IL FILE DI BACKUP DEL DATABASE LOCALE

3. **Ripristino su RDS**: La migrazione è stata finalizzata eseguendo una procedura direttamente sull'istanza cloud:

```
exec msdb.dbo.rds_restore_database
@restore_db_name = 'todo-db',          -> nome del db su AWS
@s3_arn_to_restore_from ='arn:aws:s3:::tododbbackup/ToDoDB.bak';    -> Endpoint del file .bak
```

FIGURA 40: QUERY ESEGUITA PER FARE LA MIGRAZIONE DEI DATI DAL DATABASE LOCALE ALL'ISTANZA RDS

Infine lo stato dell'operazione è stato monitorato tramite il comando *rds_task_status* fino al completamento della migrazione.



	task_id	task_type	database_name	% complete	duration(mins)	lifecycle	task_info	last_updated	created_at	S3_object_arn	overwrite_S3_backup_file
1	15	BACKUP_DB	TestDB	0	1	CREATED	NULL	2019-09-21 16:05:58.770	2019-09-21 16:05:58.770	arn:aws:s3:::rdstestsql/TestDB_Full.bak	1

	task_id	task_type	database_name	% complete	duration(mins)	lifecycle	task_info	last_updated	created_at	S3_object_arn
1	15	BACKUP_DB	TestDB	100	2	IN_PROGRESS	[2019-09-21 16:06:33.880] Task execution has st...	2019-09-21 16:06:45.190	2019-09-21 16:05:58.770	arn:aws:s3:::rdstestsql/TestDB_Full.ba

	task_id	task_type	database_name	% complete	duration(mins)	lifecycle	task_info	last_updated	created_at	S3_object_arn
1	15	BACKUP_DB	TestDB	100	2	SUCCESS	[2019-09-21 16:06:33.880] Task execution has st...	2019-09-21 16:07:27.190	2019-09-21 16:05:58.770	arn:aws:s3:::rdstestsql/TestDB_Full.bak

FIGURA 41: OUTPUT DEL COMANDO "RDS_TASK_STATUS" DURANTE LA MIGRAZIONE (IMMAGINE PRESA DA HTTPS://BLOG.CSDN.NET/CULUO4781/ARTICLE/DETAILS/107618452)

Ottimizzazione delle performance e accessibilità: Inizialmente, l'istanza RDS è stata resa accessibile solo all'IP dello sviluppatore per le verifiche strutturali tramite SSMS.
In fase di esercizio, l'accessibilità è stata ottimizzata limitando le connessioni in entrata esclusivamente al *Security Group* del backend (ECS). Questo garantisce che il database sia "invisibile" a internet, ma altamente performante per l'applicazione, che risiede nella stessa regione AWS. Inoltre, la stringa di connessione nel backend è stata momentaneamente aggiornata puntando all'Endpoint di RDS, per poter testare che il database sia funzionante prima di passare al resto dell'infrastruttura.

4.3 Strategie di sicurezza: IAM Roles e il principio di "Least Privilege[32]"

La sicurezza è stata implementata seguendo il principio di *Least Privilege*, assicurando che ogni componente abbia solo i permessi strettamente necessari.

- **Execution Roles**: Sono stati creati ruoli specifici per consentire ai task di ECS di scrivere i log su CloudWatch e di recuperare le immagini da ECR.
- **Policy Granulari**: Invece di assegnare permessi amministrativi generici, ogni servizio ha ricevuto policy JSON limitate alle sole azioni indispensabili (es. s3:PutObject solo su uno specifico bucket), riducendo al minimo la superficie di attacco in caso di compromissione di un componente.
- **IAM Task Role** (Permessi applicativi): Ruolo che definisce cosa può fare il codice dentro il container o la funzione una volta avviati.

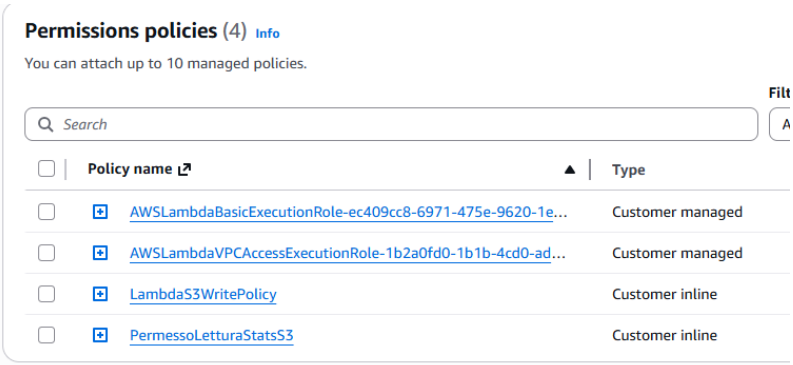


FIGURA 42: POLICY DELLA LAMBDA

5 - Containerizzazione (Il backend)

Per rendere l'applicazione To-Do List portabile e facilmente distribuibile su AWS, è stata adottata la tecnologia dei container[33]. Questo processo permette di includere in un'unico pacchetto (container) il codice, le librerie e le configurazioni necessarie, garantendo che l'applicazione funzioni in modo identico su qualsiasi piattaforma, tra cui i server di Amazon.

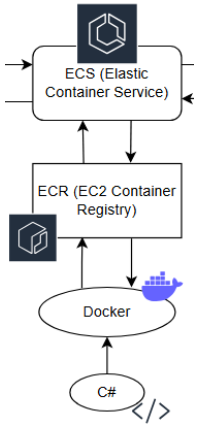


FIGURA 43: BACKEND NELL'ARCHITETTURA

5.1 Creazione dell'immagine Docker e storage su Amazon ECR

La prima fase di migrazione del backend su Amazon ha riguardato la creazione di un *Dockerfile*. Attraverso questo file, è stata definita un'immagine che compila il codice sorgente in modo ottimizzato e leggero.

Dockerfile

```
# Vedere https://aka.ms/customizecontainer per informazioni su come personalizzare il contenitore di

# Questa fase viene usata durante l'esecuzione da Visual Studio in modalità rapida (impostazione prec
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS base
USER $APP_UID
WORKDIR /app
EXPOSE 8080


# Questa fase viene usata per compilare il progetto di servizio
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
ARG BUILD_CONFIGURATION=Release
WORKDIR /src
COPY ["Backend.csproj", "."]
RUN dotnet restore "./Backend.csproj"
COPY . .
WORKDIR "/src/."
RUN dotnet build "./Backend.csproj" -c $BUILD_CONFIGURATION -o /app/build

# Questa fase viene usata per pubblicare il progetto di servizio da copiare nella fase finale
FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "./Backend.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false

# Questa fase viene usata nell'ambiente di produzione o durante l'esecuzione da Visual Studio in mod
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "Backend.dll"]
```

FIGURA 44: DOCKERFILE USATO PER CREARE IL CONTAINER CON DENTRO TUTTO IL BACKEND COMPLETO

Una volta generata l'immagine localmente, questa è stata trasferita su Amazon *Elastic Container Registry (ECR)*, che funge da repository privato e sicuro all'interno di AWS, permettendo di gestire diverse versioni dell'applicazione.

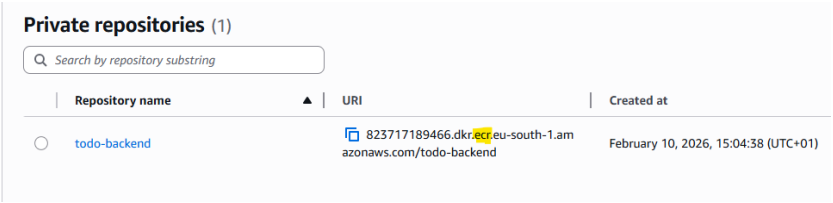


FIGURA 45: REPOSITORY ECR

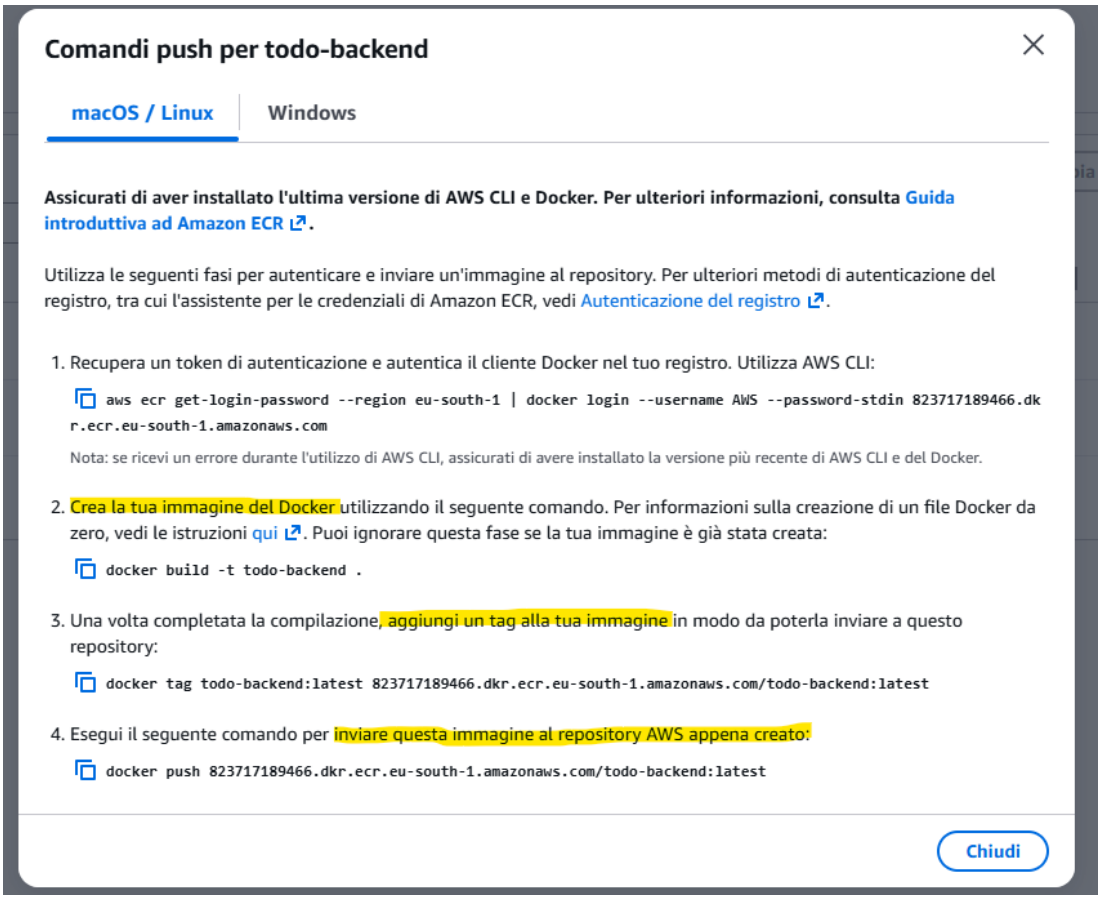


FIGURA 46: COMANDI DI PUSH FORNITI DALLA RISORSA ECR

5.2 Deployment su ECS Fargate: Task Definition e Service

Per l'esecuzione dei container è stato utilizzato Amazon *Elastic Container Service (ECS)* con il lancio di tipo *Fargate*. La scelta è stata fatta perché Fargate è un motore serverless: permette di eseguire i container senza dover gestire o aggiornare i server sottostanti.

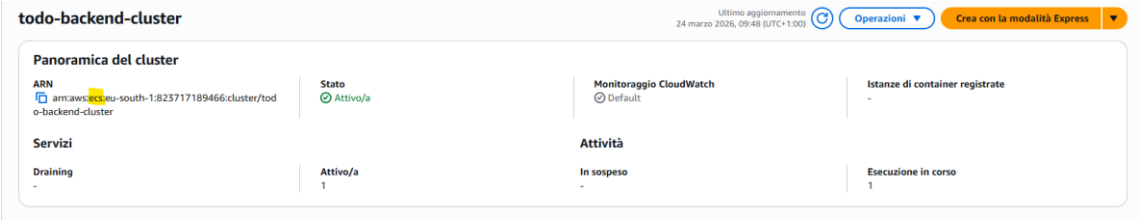


FIGURA 47: PANORAMICA DEL CLUSTER ECS

Il processo di configurazione si è diviso in due parti:

1. **Task Definition**: È stata creata una "mappatura" tecnica che descrive come il container deve essere eseguito. Qui sono stati definiti i parametri di CPU e RAM (sono state utilizzate configurazioni minime per l'ottimizzazione dei costi), le variabili d'ambiente per la connesione al database RDS e i log da inviare a CloudWatch.

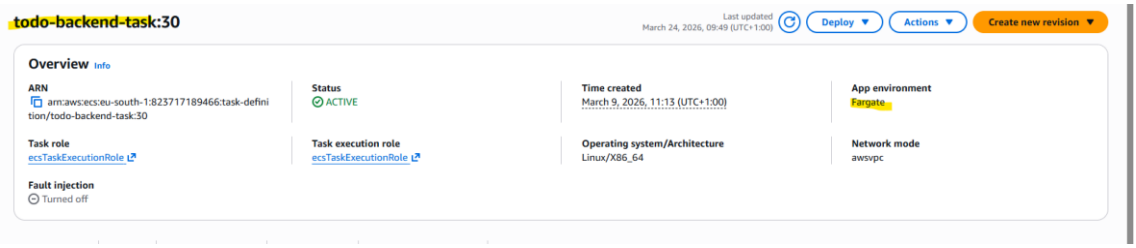


FIGURA 48: PANORAMICA DELLA TASK DEFINITION

2. **ECS Service**: È stato configurato un servizio che ha il compito di mantenere attivo il numero desiderato di task. Nel servizio è stato integrato l'Application Load Balancer (ALB), che distribuisce il traffico ai singoli task in esecuzione nelle subnet private, garantendo che, in caso di crash di uno dei container, ne venga istanziato uno nuovo.

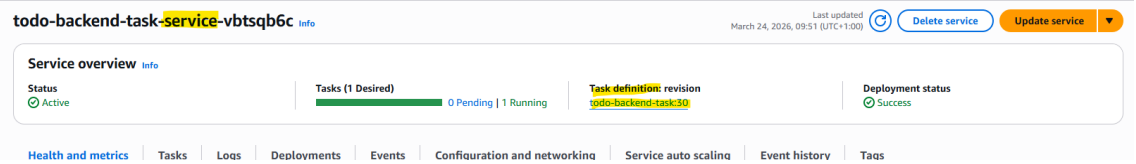


FIGURA 49: PANORAMICA DEL SERVICE

5.3 Esposizione delle API: Application Load Balancer e API Gateway

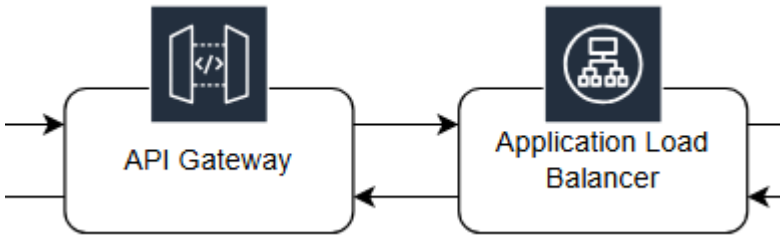


FIGURA 50: API GATEWAY E LOAD BALANCER NELL'ARCHITETTURA

Per rendere l'applicazione resiliente e sicura, il traffico in entrata dal CloudFront non entra mai direttamente ai container, ma segue un percorso strutturato che vede come protagonisti l'**API Gateway** e l'**Application Load Balancer**.

- **Application Load Balancer**: Riceve il traffico dell'API Gateway e lo distribuisce ai task eseguiti su ECS Fargate. Il suo ruolo è fondamentale per l'Alta Affidabilità: l'ALB monitora costantemente lo stato di salute dei container tramite gli *health checks*.

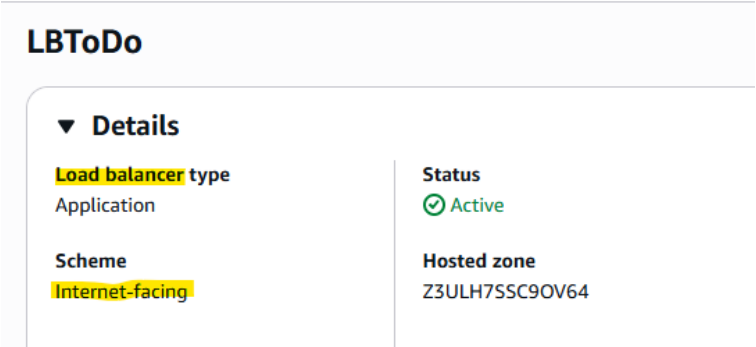


FIGURA 51: APPLICATION LOAD BALANCER

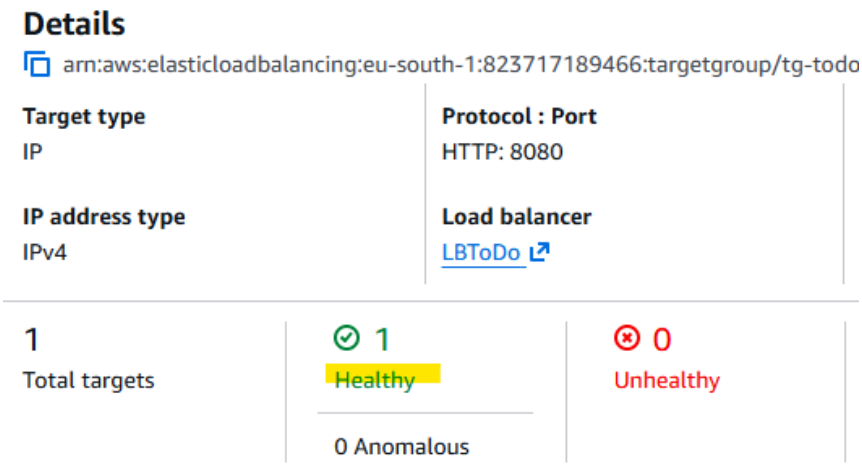


FIGURA 52: HEALTH CHECK

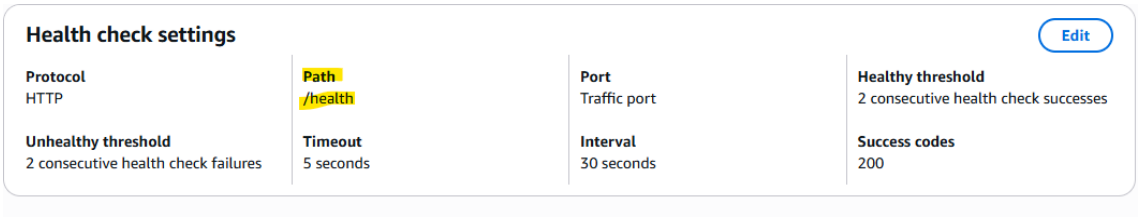


FIGURA 53: ROTTA DOVE VIENE ESEGUITO L'HEALTH CHECK

- **Amazon API Gateway**: È stato configurato come il punto di ingresso pubblico per tutte le chiamate REST provenienti dal frontend. Il suo utilizzo è strategico per diversi motivi:
  - *Astrazione*: Permette di esporre un URL pulito e unico verso l'esterno, nascondendo i dettagli dell'infrastruttura.

- *Sicurezza e Throttling*: Consente di proteggere il backend da un numero eccessivo di richieste.
- *Integrazione*: Funge da "regista" smistando le chiamate verso il Load Balancer per le funzioni CRUD[34] della To-Do List, o verso altri servizi (come per il modulo Analytics)

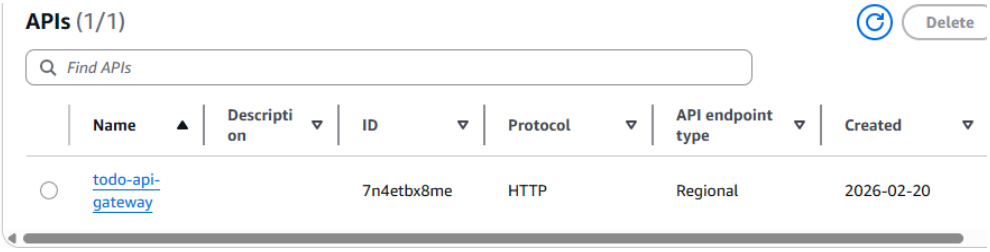


FIGURA 54: API GATEWAY

Se una task del backend dovesse fallire o riavviarsi (cambiando inevitabilmente indirizzo IP) l'ALB aggiorna automaticamente i propri Target Group.
L'applicazione è così in grado di autoripararsi e di scalare senza che l'utente finale percepisca alcuna interuzione di servizio, garantendo che il frontend punti sempre a un endpoint stabile.
In questa fase, invece di mappare singolarmente gli endpoint, si è optato per configurare una risorsa di tipo Proxy[35] (*ANY /{proxy+}*).

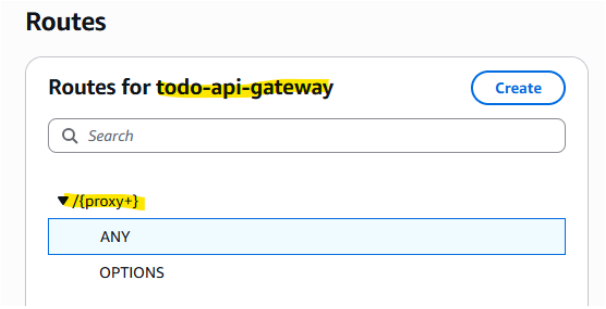


FIGURA 55: ROUTES DELL'API GATEWAY

Questa decisione è stata fondamentale per tre motivi:

- **Delegare il routing al Backend**: Poichè l'applicazione è stata interamente sviluppata e testata in ambiente locale, la logica del routing (gli endpoint come */api/login*, */api/todos*, ecc) era già definita e consolidata all'interno dei controlli .NET Core.

```
//Invoke URL di API Gateway
const API_URL = "https://7n4etbx8me.execute-api.eu-south-1.amazonaws.com/api";
```

FIGURA 56: URL DI BASE PER TUTTE LE ROTTE (INVOKE URL DI API GATEWAY)

```
        // Delete
        if (e.target.classList.contains("delete")) {

            const id = e.target.dataset.id;

            await fetch(`${API_URL}/todos/${id}`, {
                method: "DELETE",
                credentials: "include"
            });

            loadTodos();
        }
    });

    // ========================
    // LOGOUT
    // ========================
    logoutBtn.addEventListener("click", async () => {
        //richiesta POST per logout
        await fetch(`${API_URL}/login/logout`, {
            method: "POST",
            credentials: "include"
        });
        //dopo il logout ricarico la pagina
        location.reload();
    });

    // ========================
    // STATISTICHE ADMIN (Grafico)
    // ========================
    async function loadAdminStats() {
        const adminBox = document.getElementById("adminStatsBox");
        const canvas = document.getElementById("statsChart");

        if (!adminBox || !canvas) return;

        try {
            // Chiamiamo l'endpoint che legge da S3 tramite il controller C#
            const response = await fetch(`${API_URL}/todos/stats`, {
                credentials: "include"
            });
```

FIGURA 57: ESEMPI DI ROTTE DEFINITE IN BACKEND E RICHIAMATE IN FRONTEND

- **Evitare il Refactoring**: Utilizzando la risorsa Proxy, è stato possibile "trasportare" l'intera struttura delle rotte dell'API Gateway senza dover modificare una singola riga di codice lato backend o dover mappare manualmente ogni singolo endpoint sulla console AWS.
- **Agilità e Flessibilità**: È importante sottolineare che questa scelta non è vincolante. Se in una fase evolutiva del progetto si decidesse di implementare logiche di Request Transformation[36] e Validation direttamente sull'edge, API Gateway permetterebbe di dismettere il Proxy a favore di una mappatura granulare dei singoli endpoint. In questo modo, il gateway assumerebbe un ruolo attivo nel filtraggio e nel monitoraggio del traffico, dimostrando la versatilità della soluzione adottata.

6 - Distribuzione Frontend e sicurezza avanzata

Dopo aver consolidato l'architettura di backend sul cloud e averne garantito l'esposizione tramite API Gateway, il progetto si focalizza sulla fase finale: L'erogazione dell'interfaccia utente. Se finora l'attenzione è stata rivolta alla logica di elaborazione e alla persistenza dei dati, in questo capitolo viene analizzato come l'applicazione raggiunga effettivamente l'utente finale.

L'obiettivo non è solo rendere visibile la To-Do List, ma garantire che il caricamento del frontend sia rapido e configurato per comunicare in modo sicuro con le API precedentemente esposte.

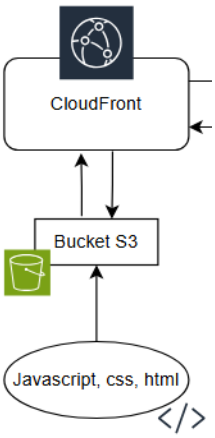


FIGURA 58: FRONTEND NELL'ARCHITETTURA

6.1 Hosting statico su S3 e distribuzione globale tramite CloudFront

I file dell'interfaccia (HTML, CSS, Javascript) sono stati caricati su un bucket Amazon S3 configurato per l'hosting di siti web statici. La distribuzione dei contenuti è affidata esclusivamente a Amazon CloudFront, la Content Delivery Network[37] (CDN) di AWS.

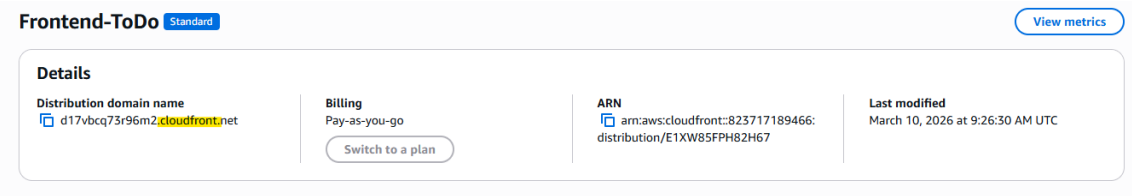


FIGURA 59: DISTRIBUZIONE DI CLOUDFRONT

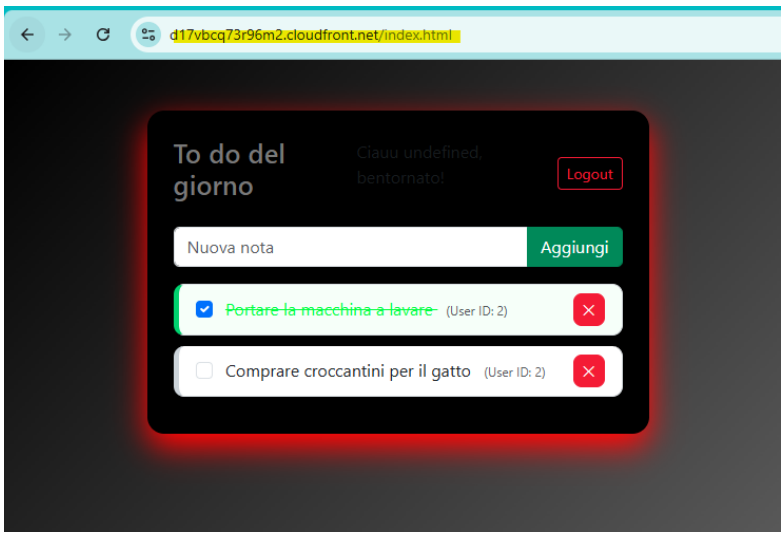


FIGURA 60: VISUALIZZAZIONE DI UN UTENTE NON-ADMIN, SITO CHE FUNZIONA SUL DISTRIBUTION DOMAIN NAME DI CLOUDFRONT

- **Content Delivery Network** (CDN): CloudFront distribuisce i file del frontend su una rete di Edge Location distribuite geograficamente. Questo riduce la latenza, poiché l'utente scarica l'applicazione dal server più vicino alla sua posizione fisica.
- **Erogazione HTTPS**: CloudFront fornisce il certificato SSL necessario per garantire che tutta la comunicazione tra il browser dell'utente e l'infrastruttura AWS sia criptata, un requisito ormai imprescindibile per qualsiasi applicazione web moderna che gestisca credenziali di accesso.

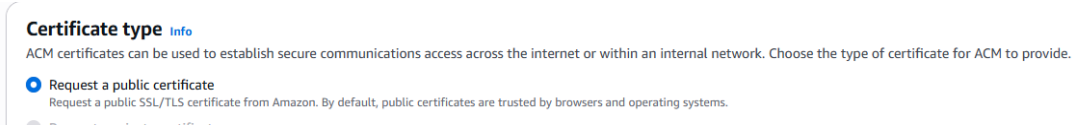


FIGURA 61: RICHIESTA DI CERTIFICATO SSL

6.2 Protezione degli accessi: Origin Access Control[38] (OAC)

Questa funzionalità permette a CloudFront di firmare crittograficamente le richieste inviate al bucket S3 ed è stata implementata per creare un perimetro di sicurezza attorno ai file statici. Grazie all'OAC, il bucket S3 accetta richieste solo se provenienti e firmate da CloudFront, impedendo a chiunque di bypassare i controlli di sicurezza della CDN.

```
{
  "Version": "2008-10-17",
  "Id": "PolicyForCloudFrontPrivateContent",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipal",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::todo-filefrontend/*",
      "Condition": {
        "ArnLike": {
          "AWS:SourceArn": "arn:aws:cloudfront::823717189466:distribution/E1XW85FPH82H67"
        }
      }
    }
  ]
}
```

FIGURA 62: BUCKET POLICY IN JSON DEL BUCKET S3 DEI FILE DI FRONTEND

6.3 Gestione dei CORS (Cross-Origin Resource Sharing)

Una volta reso disponibile il frontend su CloudFront, è stato necessario risolvere la sfida della comunicazione tra domini diversi. Poichè i file statici risiedono sulla CDN e le

chiamate API vengono indirizzate verso il gateway, il browser bloccherebbe le richieste per motivi di sicurezza, a meno di una corretta configurazione della policy CORS (Cross-Origin Resource Sharing).

In questa fase, la scelta progettuale è stata quella di mantenere la gestione dei CORS interamente lato backend, configurandola direttamente nel file *Program.cs*.

```
builder.Services.AddCors(options => {
    options.AddPolicy("AllowAll", policy => {
        policy.WithOrigins("https://d17vbcq73r96m2.cloudfront.net") //solo il sito cloudfront può chiamare l'API
            .AllowAnyMethod() //permette GET, POST, PUT, DELETE
            .AllowAnyHeader()//permette l'invio di intestazioni (es. Content-Type)
            .AllowCredentials(); // FONDAMENTALE per i cookie
    });
});
```

FIGURA 63: REGOLE DI CORS NEL FILE PROGRAM.CS NEL BACKEND

Questa decisione è stata presa per due motivi fondamentali:

- **Pre-esistenza e coerenza**: Il backend disponeva già di una logica di CORS definita durante lo sviluppo in locale. Mantenere questa gestione centralizzata nel codice ha permesso di avere un unico punto di controllo per tutte le regole di accesso.
- **Prevenzione dei conflitti**: Delegare la gestione dei CORS all'API Gateway avrebbe potuto generare conflitti tra gli header[39] inviate dal gateway a quelle inviate dal server ECS. Gestendo tutto dal backend, si è evitato il rischio di inviare "doppi header" CORS, una condizione che causerebbe il rigetto della richiesta da parte dei browser moderni, garantendo così una comunicazione fluida e priva di errori tra il sito CloudFront e API.

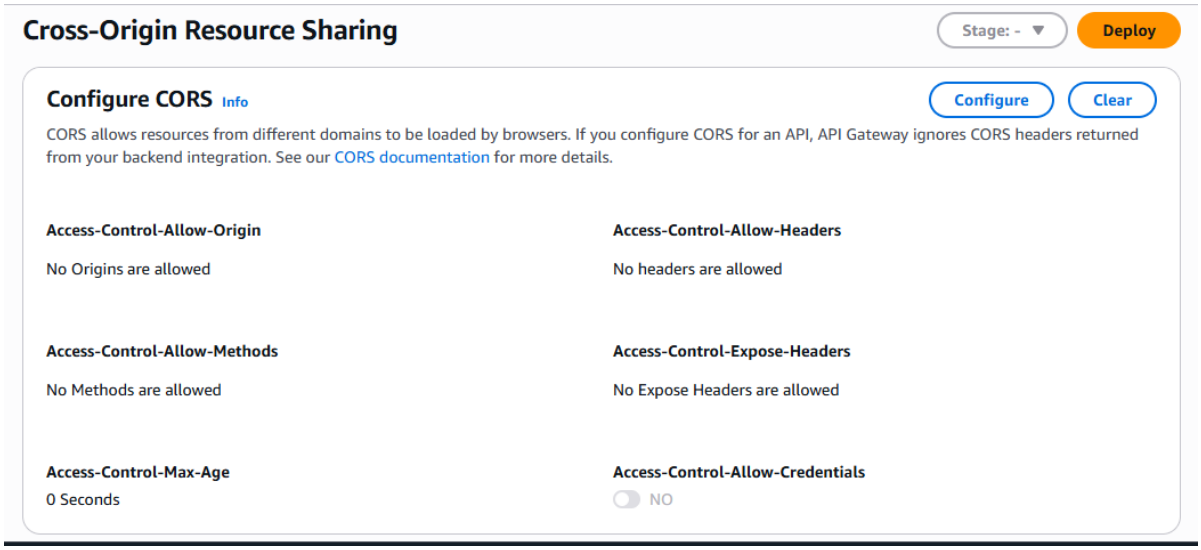


FIGURA 64: REGOLE DI CORS DELL'API GATEWAY (TUTTE DISATTIVATE PER LA GESTIONE SOLO NEL BACKEND)

Fondamentale in questo passaggio è stato l'invio delle Credentials (*.AllowCredentials()*), che permette al browser di allegare il cookie contenente il Token casuale ad ogni chiamata asincrona, completando così il ciclo di sicurezza e usabilità dell'intera piattaforma. Una volta controllare che la distribuzione CloudFront funzioni si può cambiare l'endpoint nel backend mettendo il Distribution domain name.

```
builder.Services.AddCors(options => {
    options.AddPolicy("AllowAll", policy => {
        policy.WithOrigins("https://d17vbcq73r96m2.cloudfront.net") //solo il sito c
            .AllowAnyMethod() //permette GET, POST, PUT, DELETE
            .AllowAnyHeader()//permette l'invio di intestazioni (es. Content-Type)
            .AllowCredentials(); // FONDAMENTALE per i cookie
    });
});
```

FIGURA 65: ALLOWCREDENTIALS NELLE REGOLE DI CORS LATO BACKEND

7 - Modulo Analytics e Data Automation (Serverless)

Dopo aver completato l'infrastruttura di base e aver garantito l'erogazione sicura del frontend tramite CloudFront, il progetto si è arricchito di una funzionalità avanzata dedicata alla gestione amministrativa. Questo capitolo illustrala progettazione di un sistema di analisi dati che sfrutta il paradigma Serverless, garantendo che la generazione delle statistiche avvenga in modo completamente autonomo e senza impattare sulle risorse del backend .NET Core.

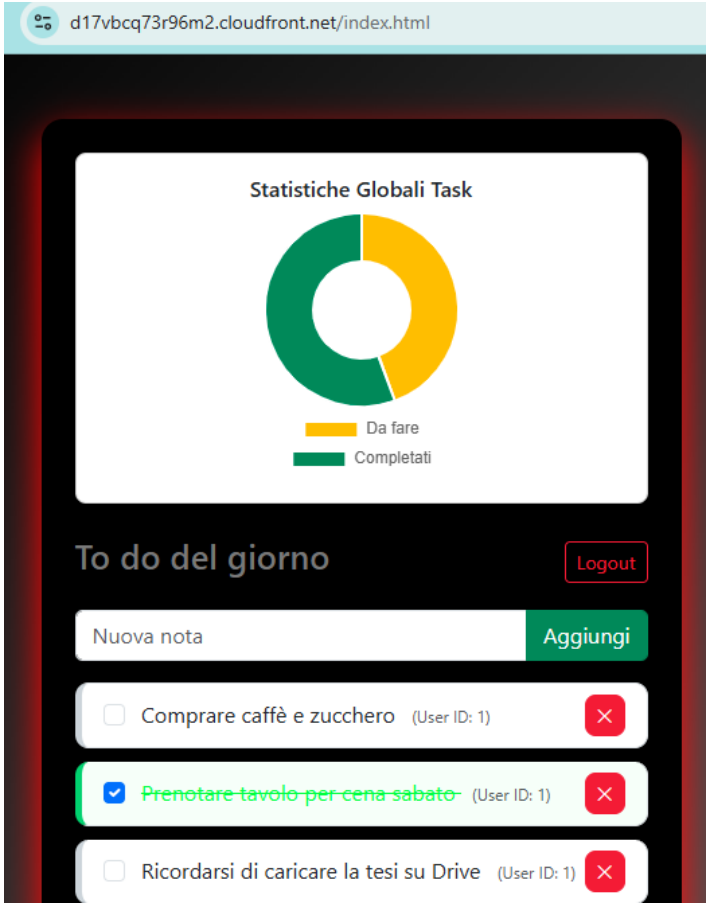


FIGURA 66: GRAFICO VISIBILE CON ACCOUNT ADMIN

7.1 Statistiche Admin in tempo reale

L'obiettivo di questo modulo è fornire all'utente con permessi di Amministratore una panoramica immediata sull'utilizzo della piattaforma, visualizzando lo stato di ogni nota, se completata o meno. Invece di eseguire la query[40] direttamente sul database ogni volta che l'Admin accede alla pagina, si è scelto di implementare una logica di pre-elaborazione dei dati.

7.2 Architettura Event-Driven[41]: AWS Lambda e Event-Bridge Scheduler[42]

Per automatizzare il processo di analisi, è stata adottata un'architettura Event-Driven. Il cuore tecnologico è rappresentato da una funzione AWS Lambda scritta in Python, scelta per la sua efficienza nella manipolazione dei dati e per la facilità di integrazione con le librerie SQL.

```
import json #libreria per trasformare i dati in formato json (per il grafico)
import boto3 #permette alla lambda di usare s3 (e altre risorse)
import pymssql  #per connetersi a sql server (rds)

def lambda_handler(event, context):
    # --- CONFIGURAZIONE del DB ---
    server = 'todo-db.c3gemowcyx3x.eu-south-1.rds.amazonaws.com'
    user = 'delia'
    password = ' '
    database = 'ToDoDB'
    bucket_name = 'todo-filefrontend' #nome del bucket s3 dove salvare il file

    try:
        conn = pymssql.connect(server, user, password, database, timeout=10)
        cursor = conn.cursor()
        query = "SELECT IsCompleted, COUNT(*) FROM Items GROUP BY IsCompleted"
        cursor.execute(query)  #invia query
        rows = cursor.fetchall()  #scarica i risultati della query in una lista "rows"
        stats = {"da_fare": 0, "completati": 0}
        for row in rows:
            if row[0] == 0: stats["da_fare"] = row[1]
            elif row[0] == 1: stats["completati"] = row[1]
        s3 = boto3.client('s3')
        s3.put_object(
            Bucket=bucket_name,
            Key='admin/stats.json', #percorso e nome del file da creare
            Body=json.dumps(stats), #trasforma il dizionario stats in testo JSON
            ContentType='application/json', #imposta che questo è un file di dati
            CacheControl='no-cache', #non si prende le cache
            ACL='public-read' #rende il file leggibile da internet/backend
        )return {
            'statusCode': 200,
            'body': json.dumps(f"Successo! Dati salvati: {stats}")
        } except Exception as e:
        print(f"ERRORE: {str(e)}")
        return {
            'statusCode': 500,
            'body': json.dumps(f"Errore tecnico: {str(e)}")
        }finally:
```

```
        if 'conn' in locals():
            conn.close()
```

CODICE 1: LAMBDA_FUNCTION.PY

- Amazon EventBridge Scheduler[42]: È stata impostata una regola di attivazione periodica (ogni 5 minuti). Questo attiva la Lambda e quest'ultima aggiorna autonomamente i dati.

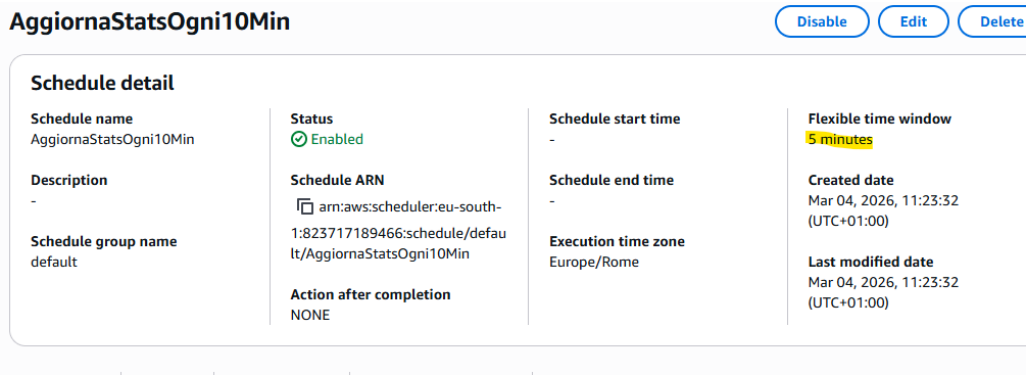


FIGURA 67: SCHEDULER DI EVENTBRIDGE - OGNI 5 MINUTI ESEGUE LA LAMBDA

- Logica di elaborazione: All'attivazione, la Lambda si connette al database RDS, esegue le query di aggregazione e formatta i risultati in un file JSON.

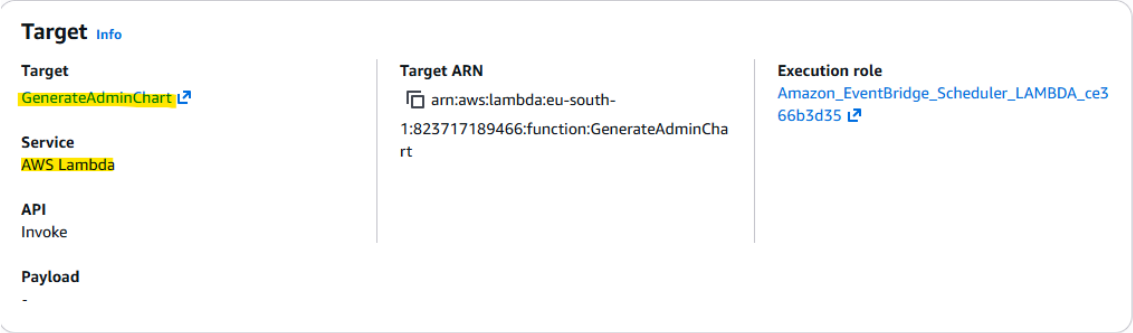


FIGURA 68: TARGET DELL'EVENTBRIDGE SCHEDULER (LA LAMBDA)

7.3 Integrazione Frontend: Visualizzazione dati tramite Chart.js[43]

Una volta che i dati analitici sono stati generati e resi disponibili, il frontend deve essere in grado di presentarli in modo intuitivo. Nel file *script.js*, è stata implementata una logica che verifica il ruolo dell'utente: se l'utente è Admin, l'applicazione effettua una chiamata per recuperare il report analitico.

Per la visualizzazione, è stata integrata la libreria Chart.js, che trasforma i dati grezzi JSON in un grafico a ciambella interattivo. Questa scelta non solo arricchisce l'interfaccia grafica, ma permette una comprensione immediata delle performance del sistema,

distinguendo nettamente l'area di lavoro degli utenti standard da quella di monitoraggio del supervisore.

```
async function loadAdminStats() {
        const adminBox = document.getElementById("adminStatsBox");
        const canvas = document.getElementById("statsChart");

        if (!adminBox || !canvas) return;
        try {
            // Chiamiamo l'endpoint che legge da S3 tramite il controller C#
            const response = await fetch(`${API_URL}/todos/stats`, {
                credentials: "include"
            });

            if (!response.ok) {
                // Se non è admin o ci sono errori, nascondiamo il box
                adminBox.classList.add("d-none");
                return;
            }

            const stats = await response.json();

            // Rendiamo visibile il contenitore
            adminBox.classList.remove("d-none");

            const ctx = canvas.getContext("2d");

            // Se esiste già un grafico (es. dopo un refresh parziale), lo distruggiamo
            if (window.myAdminChart) {
                window.myAdminChart.destroy();
            }

            window.myAdminChart = new Chart(ctx, {
                type: 'doughnut',
                data: {
                    labels: ['Da fare', 'Completati'],
                    datasets: [{
                        data: [stats.da_fare, stats.completati],
                        backgroundColor: ['#ff0707', '#00ff15'],
                        hoverOffset: 4
                    }]
                },
                options: {
                    responsive: true,
                    maintainAspectRatio: false,
                    plugins: {
                        legend: {
                            position: 'bottom'
```

```
                    }
                }
            }
        });

    } catch (error) {
        console.error("Errore nel caricamento statistiche admin:", error);
        adminBox.classList.add("d-none");
    }
```

CODICE 2: FUNZIONE NELLO SCRIPT.JS PER TRASFORMARE I DATI JSON DEL BUCKET S3 IN UN GRAFICO A TORTA

7.4 Ottimizzazione dei costi e performance tramite salvataggio su S3

L'aspetto più innovativo di questo modulo risiede nella gestione della persistenza dei report analitici. Per massimizzare le performance e ridurre al minimo i costi di esecuzione, la funzione Lambda non restituisce i dati direttamente al frontend, ma effettua il caricamento del risultato in un bucket Amazon S3 dedicato.

Il flusso tecnico si articola nel seguente modo:

- **Persistenza dei dati**: Una volta estratte le informazioni dal database RDS, la Lambda genera un file denominato *stats.json* e lo sovrascrive nel bucket S3. Questo permette di avere uno storico dell'ultima analisi effettuata sempre disponibile come risorsa statica.

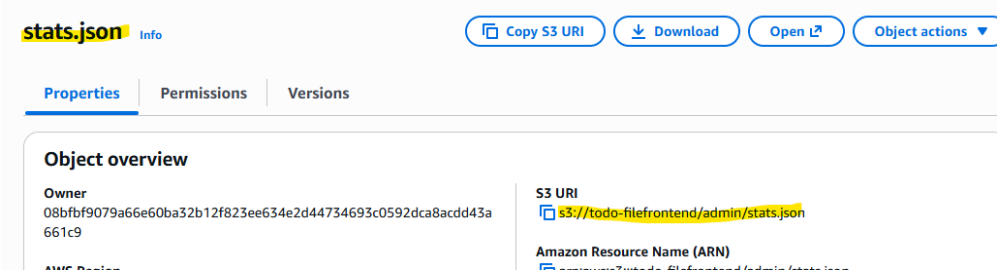


FIGURA 69: FILE DOVE VENGONO SALVATI I DATI JSON RICAVATI CON LA LAMBDA

- **Meccanismo di Caching**: Il database viene interrogato esclusivamente dalla Lambda una volta ogni 5 minuti. Il frontend scarica direttamente il file JSON dal bucket S3.
- **Efficienza economica**: Questo approccio riduce drasticamente i costi di computazione (meno esecuzioni Lambda e meno carico su RDS) e garantisce tempi di risposta immediati, poiché leggere un file da S3 è tecnicamente molto più veloce e meno oneroso che generare un report dinamico in tempo reale.

8 - Evoluzione DevOps e automazione del ciclo di vita

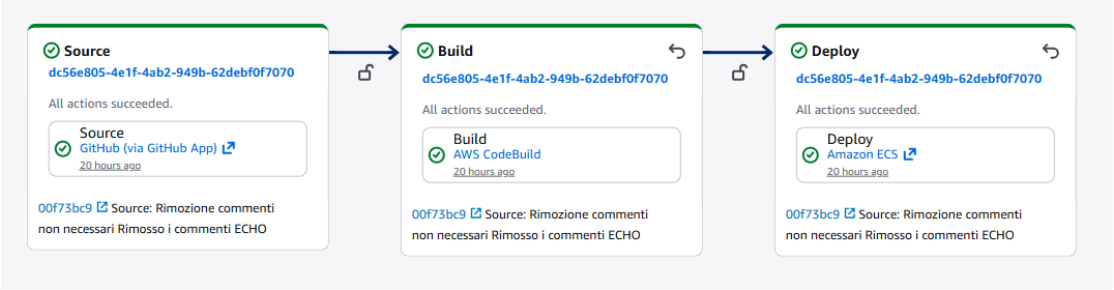


FIGURA 70: STRUTTURA DELLA PIPELINE DI AUTOMAZIONE

Inizialmente concepita come un'implementazione futura per contesti *enterprise*, l'adozione di metodologie DevOps[44] avanzate è stata integrata nel progetto per trasformare i processi manuali in flussi completamente automatizzati. L'architettura è stata evoluta superando il processo iterativo manuale (compilazione locale, upload su ECR, aggiornamento della Task Definition e del Service), eliminando il "Time to Market" manuale e minimizzando l'errore umano attraverso un approccio Infrastructure-as-Code.

8.1 Automazione del flusso CI/CD[45]

L'intero ciclo di vita del software è stato automatizzato con CodePipeline per garantire che l'ambiente Cloud sia costantemente sincronizzato con l'ultima versione stabile del codice. Il fulcro di questa automazione è il GitHub Webhook:

- **Source**: Ogni operazione di *git push* effettuata sul repository GitHub[46] scatena automaticamente la pipeline tramite Webhook, garantendo che l'ambiente Cloud sia sempre sincronizzato con l'ultima versione stabile del codice.

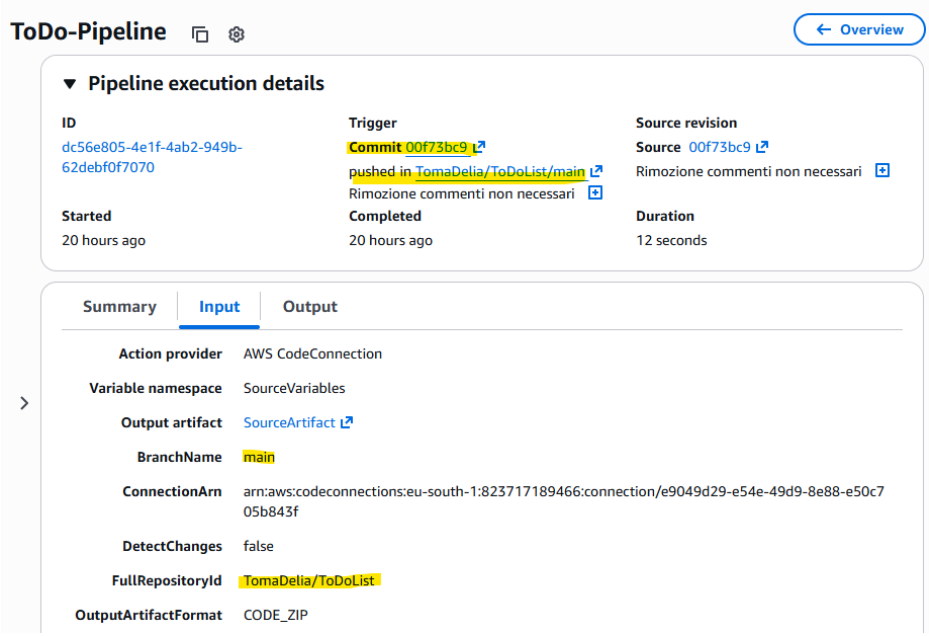


FIGURA 71: SOMMARIO DETTAGLI DEL BLOCCO "SOURCE"

- **Build & Security** (AWS CodeBuild): In questa fase, il codice .NET viene compilato e viene generata una nuova immagine Docker. Una volta validata, l'immagine viene caricata su Amazon ECR con un tag univoco (commit ID[47]).

```
[Container] 2026/03/30 13:58:27.108045 Running command printf '[{"name":"todo-backend","imageUri":"%s"}]' $REPOSITORY_URI:$IMAGE_TAG > imagedefinitions.json

[Container] 2026/03/30 13:58:27.115857 Phase complete: POST_BUILD State: SUCCEEDED
[Container] 2026/03/30 13:58:27.115870 Phase context status code:  Message:
[Container] 2026/03/30 13:58:27.227561 Expanding base directory path: .
[Container] 2026/03/30 13:58:27.231163 Assembling file list
[Container] 2026/03/30 13:58:27.231178 Expanding .
[Container] 2026/03/30 13:58:27.234380 Expanding file paths for base directory .
[Container] 2026/03/30 13:58:27.234393 Assembling file list
[Container] 2026/03/30 13:58:27.234396 Expanding imagedefinitions.json
[Container] 2026/03/30 13:58:27.237517 Found 1 file(s)
[Container] 2026/03/30 13:58:27.239805 Set report auto-discover timeout to 5 seconds
[Container] 2026/03/30 13:58:27.239853 Expanding base directory path:  .
[Container] 2026/03/30 13:58:27.243040 Assembling file list
[Container] 2026/03/30 13:58:27.243054 Expanding .
[Container] 2026/03/30 13:58:27.246233 Expanding file paths for base directory .
[Container] 2026/03/30 13:58:27.246246 Assembling file list
[Container] 2026/03/30 13:58:27.246249 Expanding **/*
[Container] 2026/03/30 13:58:27.250740 No matching auto-discover report paths found
[Container] 2026/03/30 13:58:27.250765 Report auto-discover file discovery took 0.010960 seconds
[Container] 2026/03/30 13:58:27.250782 Phase complete: UPLOAD_ARTIFACTS State: SUCCEEDED
[Container] 2026/03/30 13:58:27.250788 Phase context status code:  Message:
```

FIGURA 72: SOMMARIO ULTIMO "BUILD" DI PROVA (ESEMPIO: ULTIME DUE FASI ANDATE A BUON FINE)

- **Deploy** (ECS Rolling Update): La pipeline comunica con Amazon ECS per avviare l'aggiornamento del servizio, coordinando la sostituzione dei container attivi.

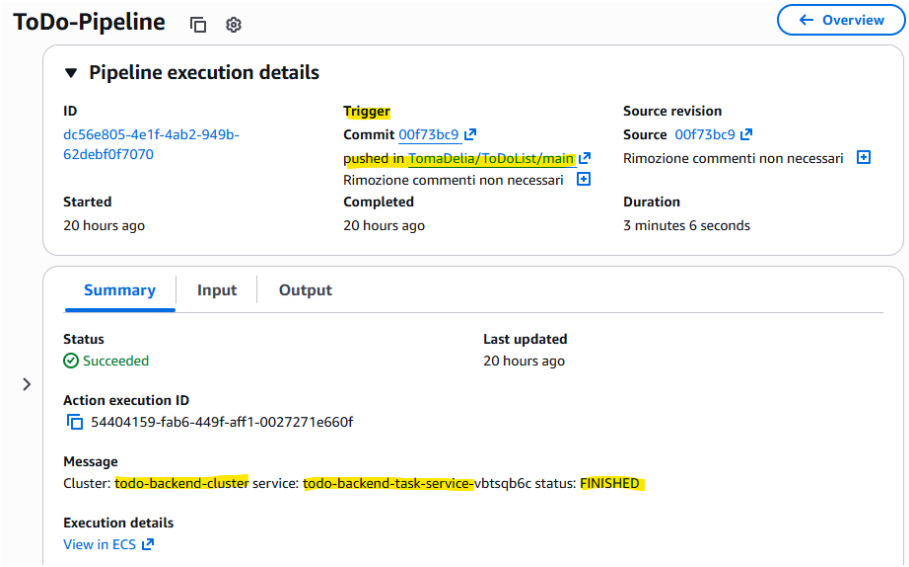


FIGURA 73: SOMMARIO DELL'ULTIMO RISULTATO DEL BLOCCO "DEPLOY"

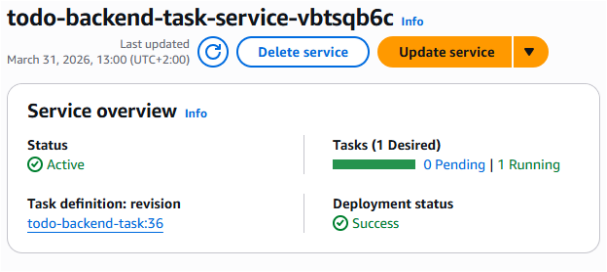


FIGURA 74: SERVICE IN FUNZIONE CON TASK DEFINITION CREATA AUTOMATICAMENTE DALLA PIPELINE

8.2 Build Specification e tracciabilità

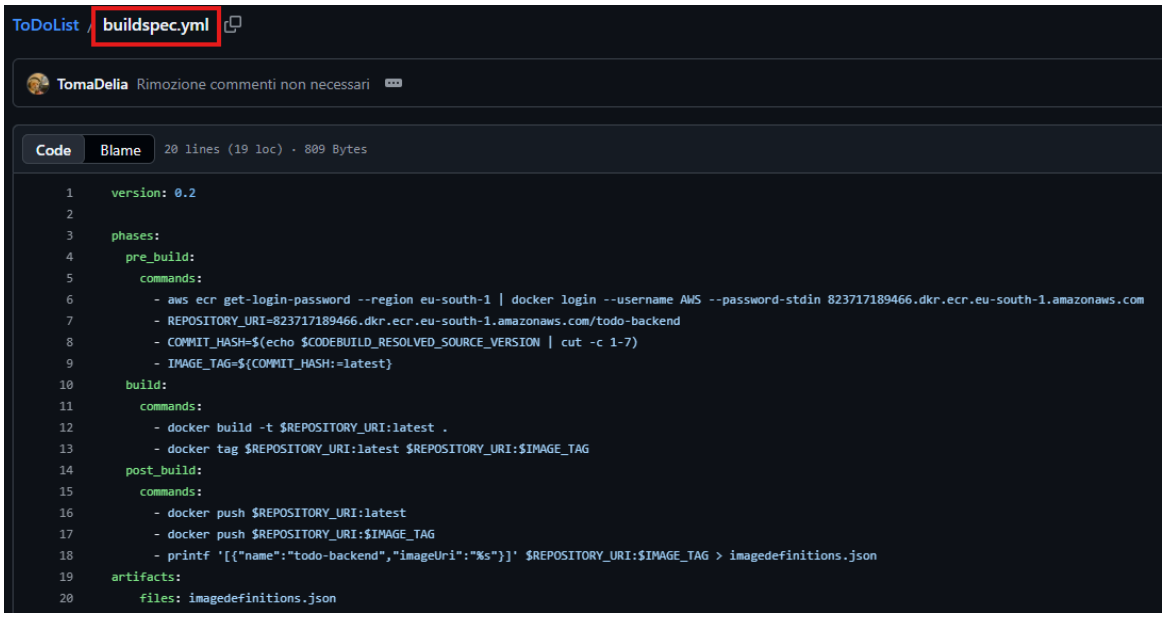


FIGURA 75: FILE BUILDSPEC.YML

La fase di build, gestita tramite un file di configurazione (buildspec.yml), non si limita alla semplice compilazione del codice .NET. Per garantire una tracciabilità granulare, è stata implementata una logica di Dynamic Tagging[48] basata sullo Short-SHA[49] (vengono usati i primi 7 caratteri del commit ID).

```
- COMMIT_HASH=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c 1-7)
- IMAGE_TAG=${COMMIT_HASH:=latest}
```

FIGURA 76: ESTRAZIONE DELLO SHORT-SHA DAL COMMIT GITHUB PER IL TAGGING UNIVOCO DELL'IMMAGINE DOCKER

L'immagine Docker non viene semplicemente sovrascritta, ma caricata su Amazon ECR con questo tag univoco. Questa tecnica permette di correlare ogni container in esecuzione a una specifica riga di codice sorgente, facilitando operazioni di auditing o debugging: ogni artefatto nel registro diventa così un'istantanea immutabile e identificabile della storia del progetto.

```
commands:
  - docker push $REPOSITORY_URI:latest
  - docker push $REPOSITORY_URI:$IMAGE_TAG
```

FIGURA 77: COMANDO PER IL CARICAMENTO DELL'IMMAGINE APPENA CREATA SU ECR

8.3 Deployment Resiliente e Zero-Downtime

L'integrazione con ECS Fargate permette di gestire il rilascio attraverso un Rolling Update intelligente. Il sistema avvia i nuovi container e attende che l'Application Load Balancer ne confermi lo stato di salute (Health Check). Solo dopo la convalida, il traffico viene deviato dai vecchi ai nuovi container, garantendo la continuità del servizio (Zero-Downtime). La conservazione della storicità delle immagini in ECR, legata ai commit ID, assicura inoltre la possibilità di effettuare un rollback immediato a una versione precedente in caso di anomalie post-rilascio.

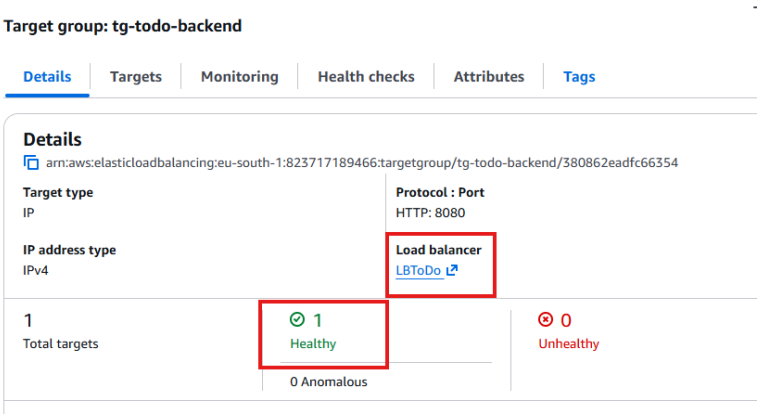


FIGURA 78: HEALTH CHECK LOAD BALANCER

9 - Conclusioni e Sviluppi Futuri

Il percorso di migrazione e modernizzazione dell'applicazione To-Do List ha permesso di trasformare un prototipo locale in un'infrastruttura Cloud-Native resiliente e sicura. Il lavoro svolto ha rappresentato una re-ingegnerizzazione dei processi di erogazione e analisi dei dati, gettando basi per un sistema pronto a scalare in un contesto di produzione reale.

9.1 Analisi dei risultati ottenuti

L'obiettivo primario di trasferire l'intera logica applicativa su AWS è stato raggiunto. L'architettura finale garantisce:

- **Isolamento e sicurezza**: Con la separazione tra subnet pubbliche e private e all'uso di CloudFront con le regole di Origin-Access Control (OAC), i dati sensibili su RDS e i container su ECS non sono mai esposti direttamente a internet.
- **Automazione DevOps**: L'implementazione di una pipeline CI/CD completa ha eliminato i processi di deployment manuale. Ogni aggiornamento del codice viene ora testato, containerizzato e distribuito automaticamente, garantendo una sincronizzazione costante tra repository e infrastruttura.
- **Resilienza e Zero-Downtime**: L'integrazione tra l'Application Load Balancer e il servizio ECS Fargate assicura la continuità del servizio; il sistema è in grado di rilevare le task del service malfunzionanti e sostituirli automaticamente eseguendo un Rollback alla task funzionante senza interruzioni per l'utente.
- **Efficienza Serverless**: Il modulo per l'analitica dei dati dimostra come sia possibile estrarre valore dai dati senza gravare sui costi fissi, con l'attivazione delle risorse di calcolo (Lambda) solo per il tempo strettamente necessario.

9.2 Evoluzione verso l'Infrastruttura IaC

Nonostante l'attuale stabilità dell'infrastruttura, il passaggio a un contesto di produzione enterprise richiederebbe un ulteriore implementazione nella gestione delle risorse. L'obiettivo è minimizzando l'errore umano attraverso una gestione "Infrastracture-as-Code".

Infrastructure as Code (IaC): Per garantire la scalabilità e la replicabilità dell'intero ecosistema, il passaggio successivo sarebbe la migrazione della gestione infrastrutturale verso strumenti di Infrastructure as Code come Terraform o AWS CloudFormation.

- **Versionamento della rete**: Definire VPC, Subnet Security Group e policy IAM tramite codice permette di trattare l'infrastruttura come se fosse software, mantenendo uno storico delle modifiche su AWS CodeCommit[50].
- **Ambienti effimeri**: L'Iac permetterebbe di creare e distruggere interi ambienti di "Test" o "Staging" identici a quello di produzione in pochi minuti, assicurando che ogni nuova funzionalità venga testata in un ambiente speculare a quello reale, eliminando errori dovuti a configurazioni manuali differenti.

9.3 Monitoraggio e osservabilità avanzata

In un'ottica di produzione enterprise, l'"osservabilità" del sistema diventa critica. Attualmente il sistema si appoggia ai log base di CloudWatch, ma per una scalabilità reale si renderebbe necessaria l'adozione di:

- **Distributed Tracing** (AWS X-Ray): Per tracciare le richieste "end-to-end" dal frontend al database, permettendo di individuare istantaneamente eventuali colli di bottiglia[51] o latenze anomale tra API Gateway e i microservizi su ECS.
- **Dashboarding** (Amazon Managed Grafana): L'aggregazione dei dati di performance in dashboard grafiche permetterebbe all'Admin di monitorare in tempo reale lo stato delle risorse (CPU, RAM, connessioni RDS) e di ricevere alert preventivi prima che un eventuale sovraccarico impatti sugli utenti finali.

9.4 Considerazioni personali

Il passaggio dallo sviluppo in locale alla gestione di un'infrastruttura Cloud distribuita ha imposto un cambio di mentalità: non più focalizzata solo sulla scrittura di codice, ma sull'architettura, la sicurezza, l'ottimizzazione delle risorse e sui processi di deploy e integrazione continua (CI/CD).

La necessità di operare in regime di self-management, inclusa la gestione autonoma di un account AWS personale e il confronto diretto con i limiti del free tier, ha richiesto un impegno costante nel troubleshooting di problematiche sistemistiche complesse, dalla

corretta gestione dei certificati SSL[52] su CloudFront alla risoluzione dei conflitti CORS tra diversi servizi.

Vedere l'applicazione online e perfettamente funzionante non è stato solo il traguardo finale, ma la prova tangibile di aver saputo tradurre concetti teorici complessi in un'infrastruttura reale, sicura e pronta per essere scalata.

10 - Glossario termini tecnici

1 - *Big Data* → set di dati estremamente grandi e complessi che non possono essere facilmente gestiti o analizzati con gli strumenti di elaborazione dei dati tradizionali.
2 - *Architettura Client-Server* → architettura di rete nella quale genericamente un computer client o terminale si connette ad un server per la fruizione di un certo servizio.
3 - *.NET Core* → framework open source sviluppato da Microsoft, progettato per creare applicazioni multipiattaforma che possono funzionare su Windows, macOS e Linux. Supporta vari linguaggi di programmazione.
4 - *Web API* → insieme di regole e protocolli che consentono a diverse applicazioni di comunicare tra loro su Internet.
5 - *Javascript, HTML e CSS* → tre dei principali linguaggi del web lato client. HTML è lo scheletro, JavaScript è l'apparato muscolare che permette il movimento, mentre CSS è l'aspetto esteriore.
6 - *Handlebars* → Handlebars è un sistema semplice per creare pagine HTML dinamiche. Permette di mettere segnaposto nel modello (template) usando {{...}} e poi sostituirli con i dati veri in JavaScript.
7 - *SQL Server* → sistema di gestione di basi di dati relazionali (RDBMS) sviluppato da Microsoft. Permette di memorizzare, gestire e interrogare dati usando il linguaggio SQL.
8 - *Token* → è una stringa emessa da un sistema di autenticazione che prova l'identità di un utente o di un client e viene usata per autorizzare richieste senza dover ritrasmettere credenziali (username/password) ad ogni chiamata.
9 - *Scalare orizzontalmente* → aumentare la capacità di un sistema aggiungendo più macchine (istanze) allo stesso livello.
10 - *Cloud-Native* → sviluppo di applicazioni che utilizza tecnologie come microservizi e container per creare software scalabili e resilienti, progettati specificamente per funzionare nel cloud.
11 - *On-premise* → installazione e l'esecuzione di software e servizi all'interno dell'infrastruttura fisica dell'organizzazione, invece che su cloud pubblico.
12 - *Relazione uno-a-molti* → un record di una tabella è associato a più record di un'altra tabella (es. un Autore ha molti Libri; ogni Libro ha un solo Autore).

13 - *Primary key* → valore che identifica univocamente ogni riga in una tabella del database; mai nullo.
14 - *Campo booleano* → campo che memorizza solo due valori possibili.
15 - *Foreign key* → colonna in una tabella che punta alla primary key di un'altra tabella e stabilisce relazioni tra record.
16 - *Entity Framework Core* → permette di lavorare con database usando oggetti .NET invece di scrivere SQL manuale.
17 - Object-Relational Mapper → è una libreria che mette in corrispondenza oggetti del linguaggio di programmazione con record di un database relazionale, permettendo di lavorare con dati usando classi e metodi invece di SQL grezzo.
18 - *Stateless* → descrive un componente o servizio che non conserva informazioni sull'interazione passata tra richieste; ogni richiesta è indipendente.
19 - *Cookie* → piccola porzione di dati memorizzata dal browser e inviata automaticamente al server nelle richieste; usata per sessioni, preferenze e tracking.
20 - *JWT* → JSON Web Token, formato compatto per trasmettere informazioni autenticabili tra due parti; usato per autenticazione/authorization, contiene dati come *user_id* ed *exp*, verificabile senza consultare lo stato del server.
21 - *Attacchi XSS* → attacco che sfrutta l'inserimento di script malevoli in pagine web visualizzate da altri utenti; lo script viene eseguito nel browser vittima e può rubare cookie, token, o manipolare il DOM.
22 - *LocalStorage* → memoria locale nel browser che salva coppie chiave/valore persistenti per dominio; persiste oltre la sessione e non viene inviata al server automaticamente.
23 - *Cross-Origin Resource Sharing* → CORS, meccanismo che controlla quale origine può richiedere risorse da un altro dominio; il server fornisce intestazioni HTTP per permettere o negare l'accesso cross-origin.
24 - *AllowCredentials()* → impostazione CORS che indica se il server consente l'invio di credenziali (cookie, header Authorization, TLS client certificates) nelle richieste cross-origin.
25 - *Bootstrap* → framework CSS open source per creare interfacce responsive e componenti UI pronti.
26 - *Interfaccia responsive* → design che adatta layout e contenuti alla dimensione e orientamento dello schermo.

27 - *Dati JSON* → formato testo strutturato in coppie chiave/valore.

28 - *Fetch API* → API JavaScript moderna per fare richieste HTTP asincrone dal browser.

29 - *Subnet* → porzione di rete IP derivata da una rete più grande.

30 - *Classe t3.micro* → tipo di istanza AWS della famiglia T3 (burstable), con 2 vCPU virtuali basate su Intel/AMD e 1 GiB di RAM

31 - *Option Group* → contenitore di configurazioni opzionali applicabili a un'istanza DB per abilitare funzionalità aggiuntive.

32 - *Principio di Least Privilege* → principio di sicurezza che assegna agli utenti o processi solo i permessi minimi necessari per svolgere le loro funzioni.

33 - *Container* → unità leggera e isolata che include un'applicazione e tutte le sue dipendenze.

34 - *Funzioni CRUD* → operazioni Create, Read, Update, Delete usate per gestire le risorse in un'applicazione.

35 - *Risorsa Proxy* → componente che fa da intermediario tra client e server, inoltrando richieste e risposte.

36 - *Request Transformation* → modifica automatica delle richieste HTTP in transito per adattarle a backend o policy.

37 - *Content Delivery Network* → rete di server distribuiti che memorizza e distribuisce copie di contenuti vicino agli utenti per ridurre latenza, migliorare prestazioni e ridurre carico sull'origin.

38 - *Origin Access Control* → restrizione selettiva dell'accesso a dati, applicazioni e risorse.

39 - *Header* → valore inviato nelle richieste o risposte HTTP per trasmettere meta-informazioni.

40 - *Query* → richiesta di dati al database.

41 - *Architettura Event-Driven* → i componenti comunicano reagendo a eventi.

42 - *Event-Bridge* Scheduler → servizio AWS per pianificare e invocare automaticamente target secondo regole di tempo.

43 - *Chart.js* → libreria JavaScript per disegnare grafici interattivi.

44 - *DevOps* → Questa metodologia promuove la collaborazione tra i team di sviluppo e operativi, facilitando un processo di distribuzione più rapido e continuo.

45 - *CI/CD* → Continuous Integration/Continuous Delivery, automatizza il processo di integrazione e distribuzione del codice.

46 - *repository GiHub* → spazio remoto che ospita il codice sorgente e la cronologia Git di un progetto.

47 - *Commit ID* → identificatore univoco di una singola revisione in Git.

48 - *Dynamic Tagging* → applicare tag a contenuti o dati in modo dinamico, in base a valori che possono cambiare nel tempo.

49 - *Short-SHA* → versione abbreviata di un valore hash. Un hash è il risultato di una funzione crittografica o non crittografica che trasforma dati di lunghezza arbitraria in una stringa di lunghezza fissa.

50 - *CodeCommit* → servizio AWS per repository Git gestiti.

51 - *Colli di bottiglia* → componente o risorsa che rallenta l'intero sistema perché non riesce a processare il traffico.

52 - *Certificati SSL* → certificato digitale che abilita HTTPS autenticando il server e cifrando il traffico.

11 - Bibliografia e Sitografia

A

- Amazon Web Services (05/01/2024) Amazon RDS for Microsoft SQL Server. https://aws.amazon.com/it/rds/sqlserver/
- Amazon Web Services (15/01/2024) Amazon Virtual Private Cloud Documentation https://docs.aws.amazon.com/vpc/
- Amazon Web Services (18/12/2023) Using AWS Lambda with Amazon S3. https://docs.aws.amazon.com/lambda/latest/dg/with-s3.html
- Auth0 (14/11/2023) JSON Web Token (JWT) Introduction. https://auth0.com/docs/secure/tokens/json-web-tokens

C

- Create private integrations for HTTP APIs in API Gateway https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-develop-integrations-private.html

D

- Docker Inc. (05/11/2023) Docker Overview. https://docs.docker.com/get-started/docker-overview/

M

- Michael Wittig, Andreas Wittig: Amazon Web Services in Action - 2018 - Shelter Island: Manning Publications.
- Microsoft (10/02/2024) ASP.NET Core Web API documentation. https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-10.0
- Mozilla Developer Network (12/03/2024) Using HTTP cookies. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies

O

- OWASP Foundation (01/01/2024) Cross-Site Scripting (XSS) Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

J

- Jonathan Skeet: C# in Depth - 2019 - Shelter Island: Manning Publications.